

CLASSICAL VIEWING

Lecturer: Dr. Ufuk Çelikcan

Based on the slides by: E. Angel and D. Shreiner

Objectives

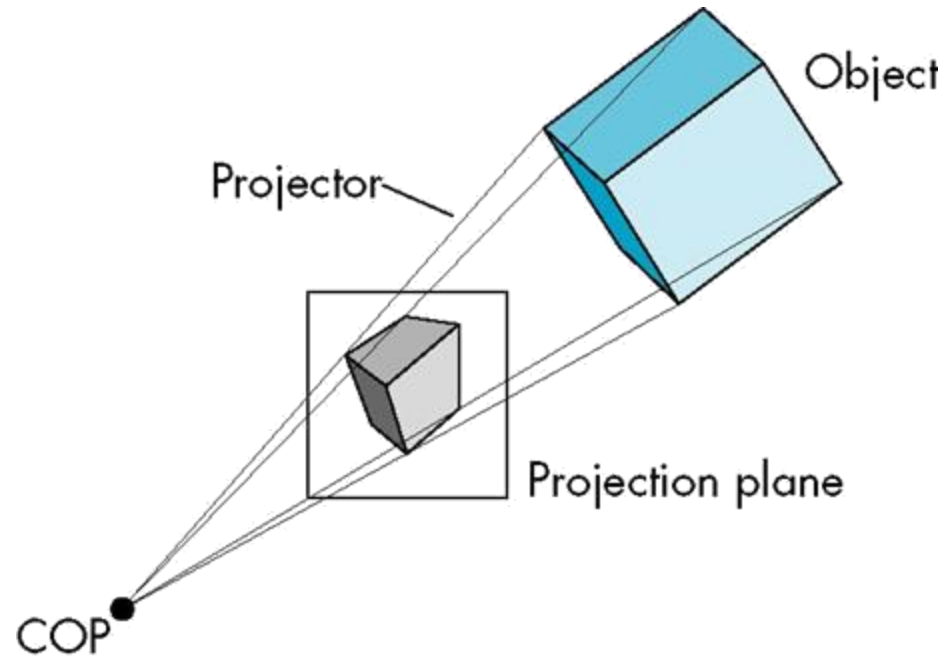
- Introduce the classical views
- Compare and contrast image formation by computer with how images have been formed by architects, artists, and engineers
- Learn the benefits and drawbacks of each type of view

Classical Viewing

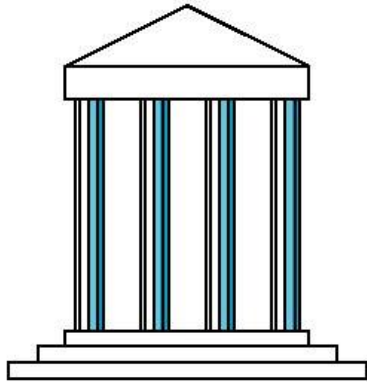
- Viewing requires three basic elements
 1. **Object(s)**
 2. A **viewer** with a projection surface
 3. **Projectors** that go from the object(s) to the projection surface
- Classical views are based on the relationship among these elements
- Each object is assumed to be constructed from flat *principal faces*
 - Buildings, polyhedra, manufactured objects

Planar Geometric Projections

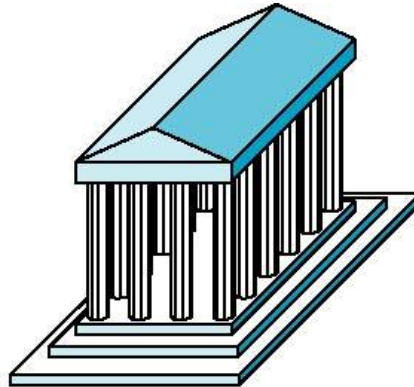
- Standard projections project onto a plane
- **Planar projectors** are lines that either
 1. converge at a center of projection
 2. are parallel
- Planar projections preserve lines
- but not necessarily angles
- **Nonplanar projections** are needed for applications such as **map construction**



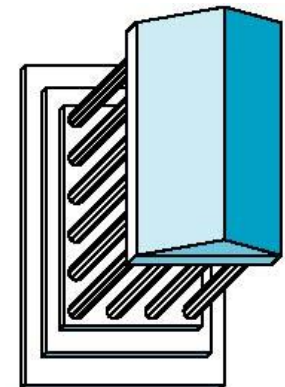
Classical Projections



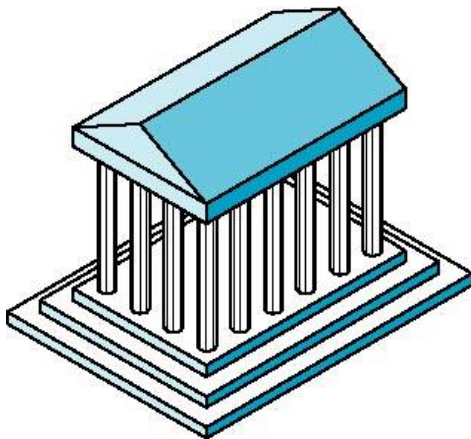
Front elevation



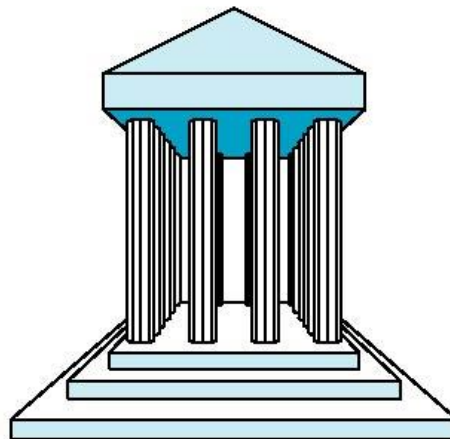
Elevation oblique



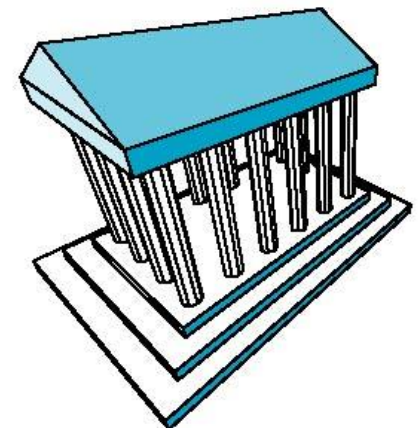
Plan oblique



Isometric



One-point perspective

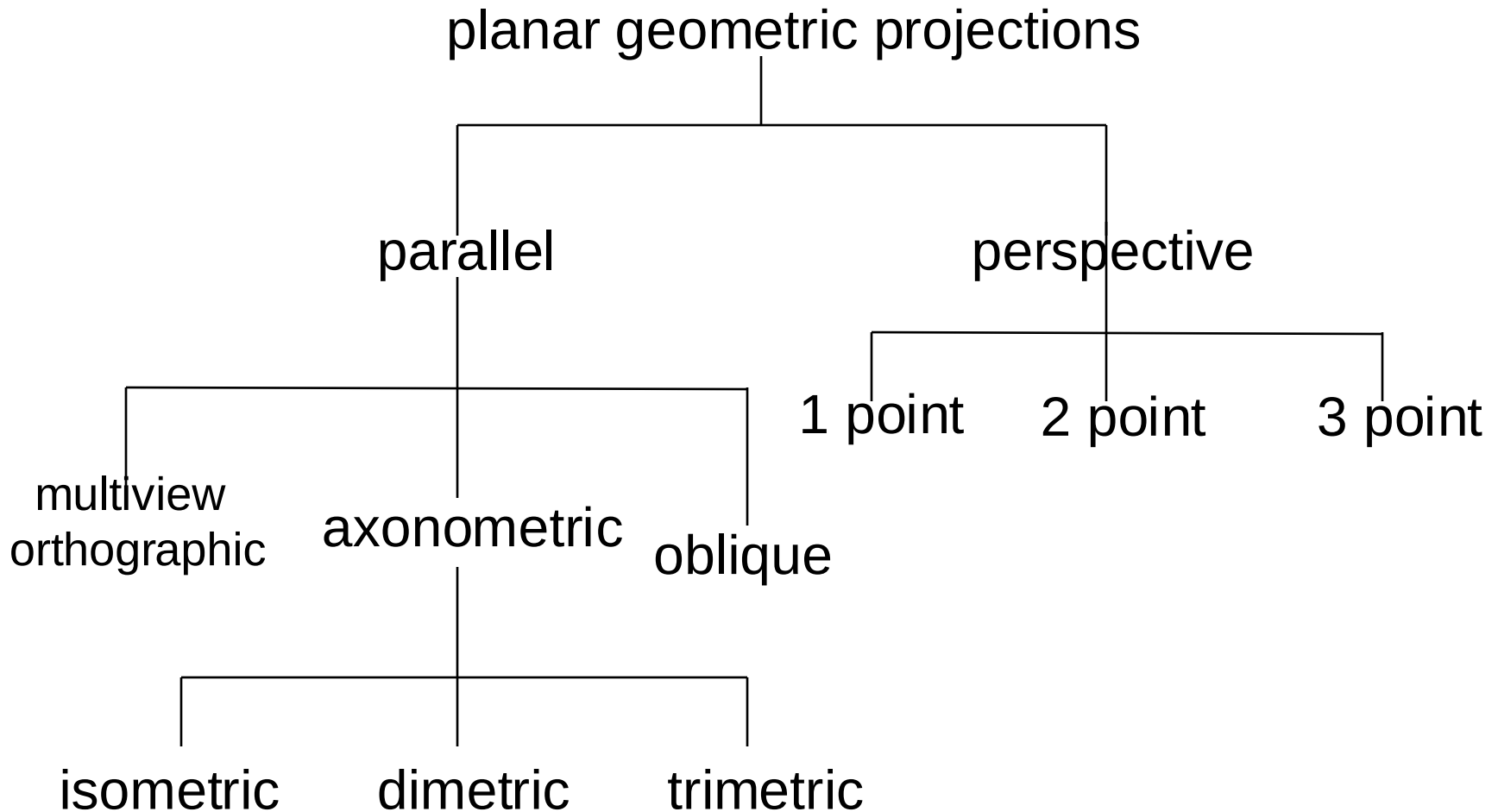


Three-point perspective

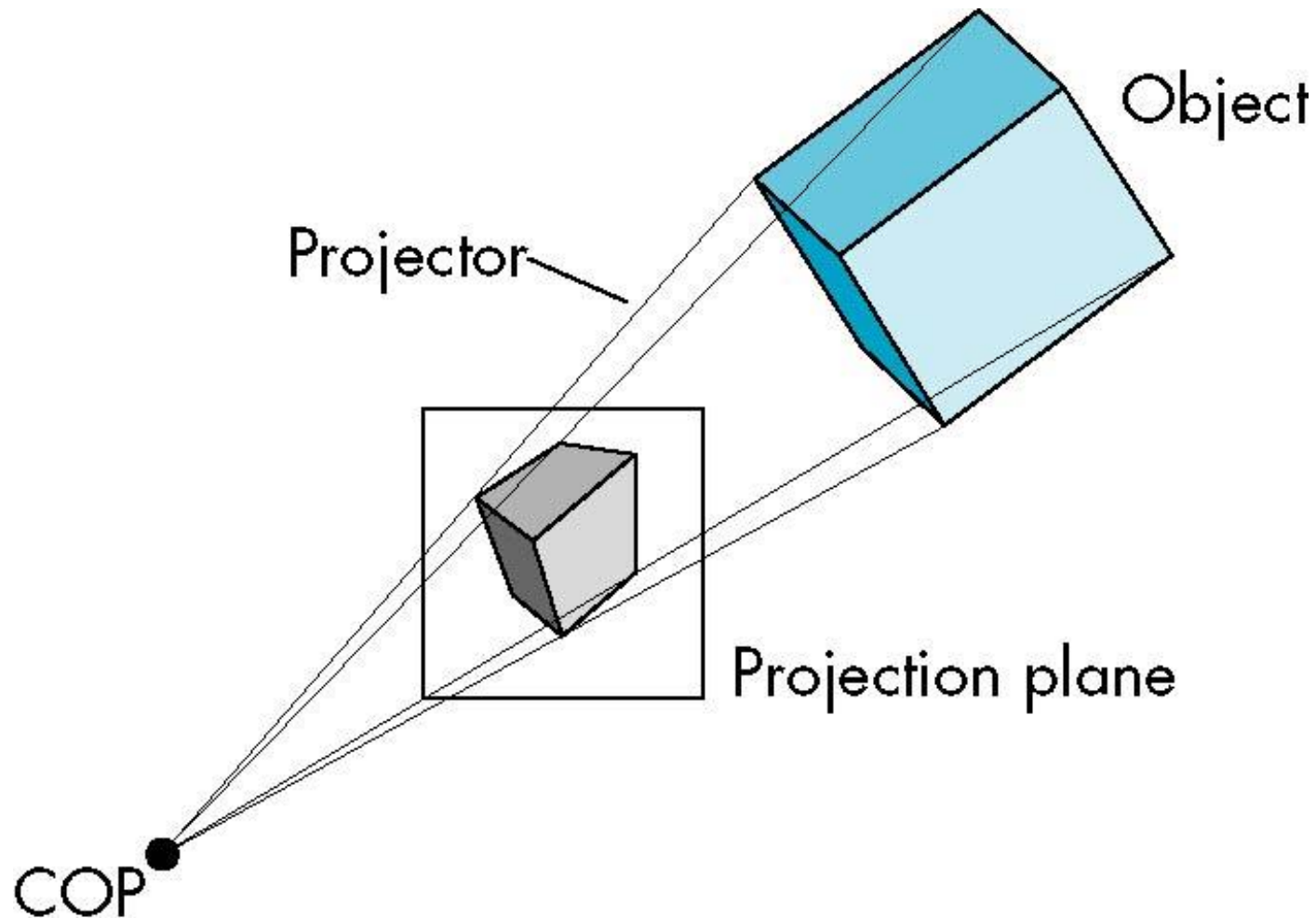
Perspective vs Parallel

- Computer graphics treats all projections the same and implements them with the same pipeline
- Classical viewing developed different techniques for drawing each type of projection
- Fundamental distinction is between parallel and perspective viewing even though mathematically: parallel viewing is the limit of perspective viewing

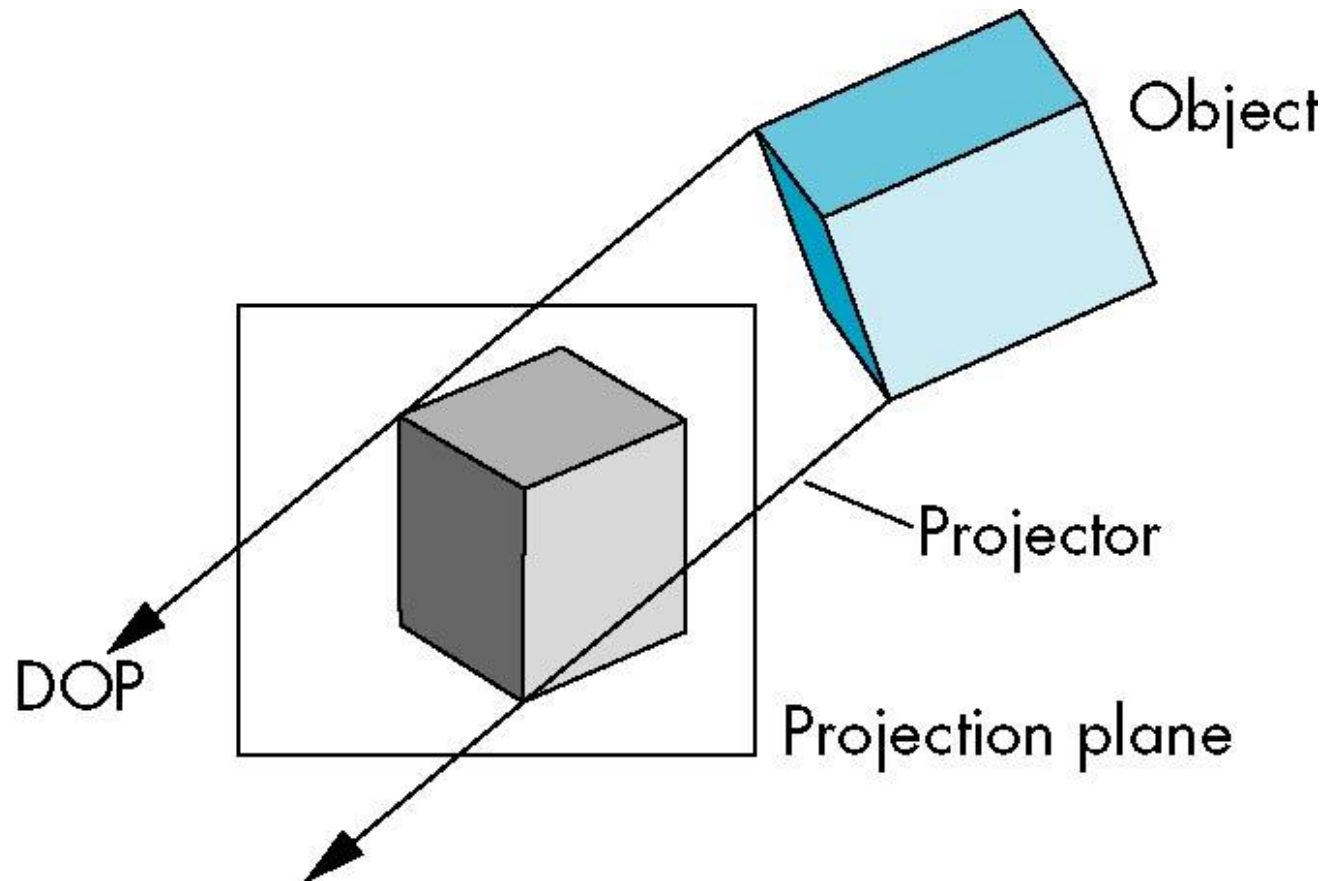
Taxonomy of Planar Geometric Projections



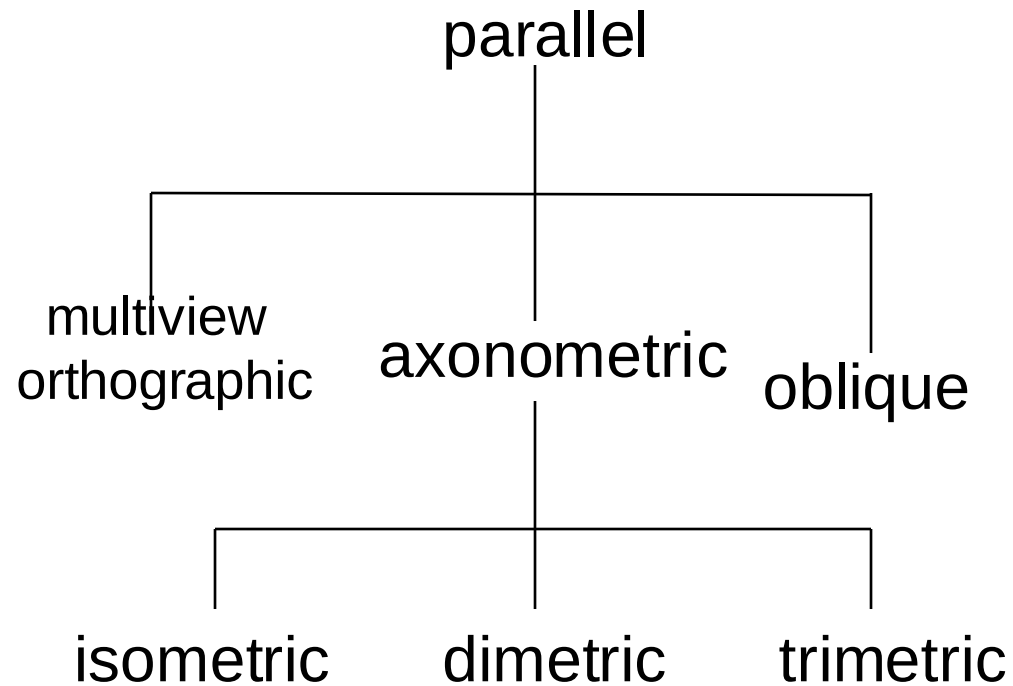
Perspective Projection



Parallel Projection

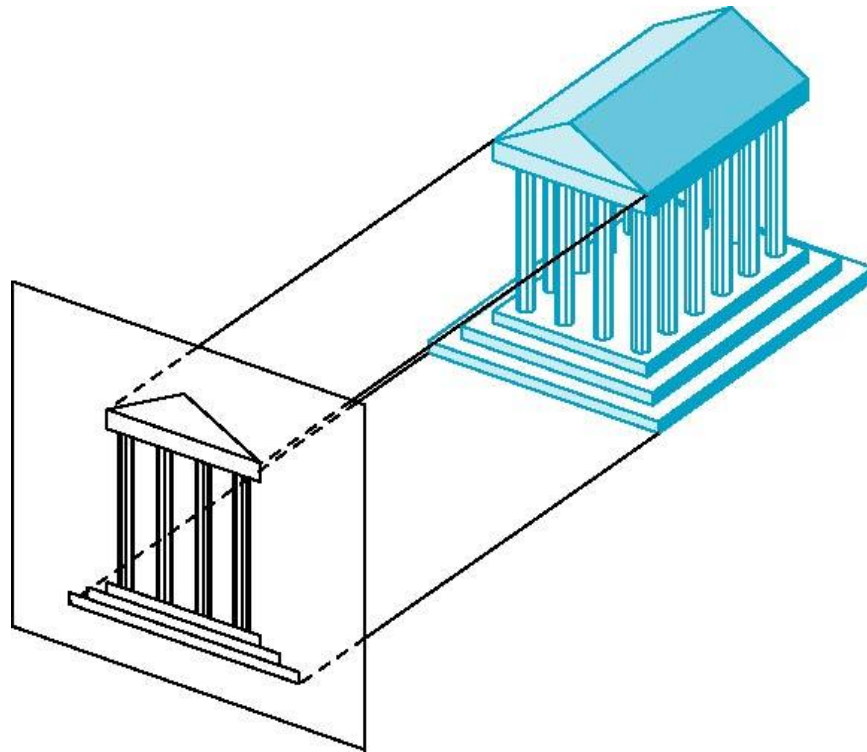


Parallel Projections



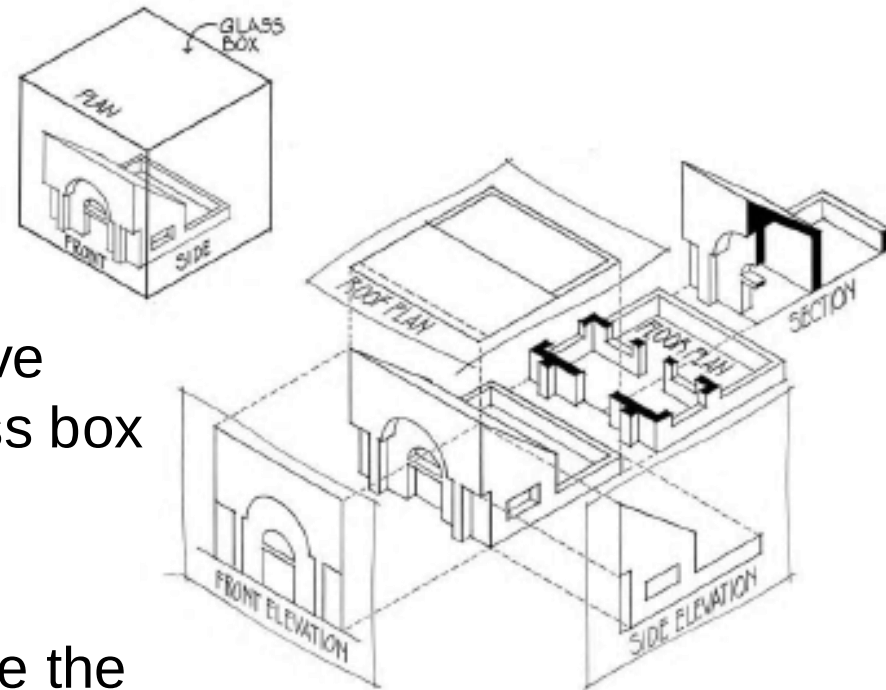
Orthographic Projection

- Projectors are orthogonal to projection surface



Orthographic Projection

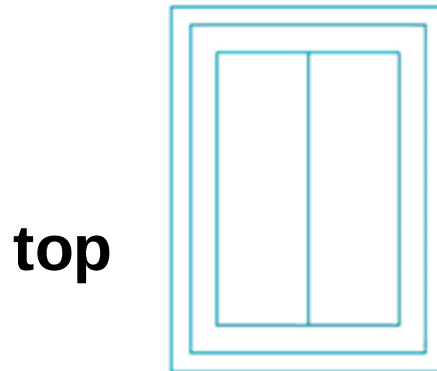
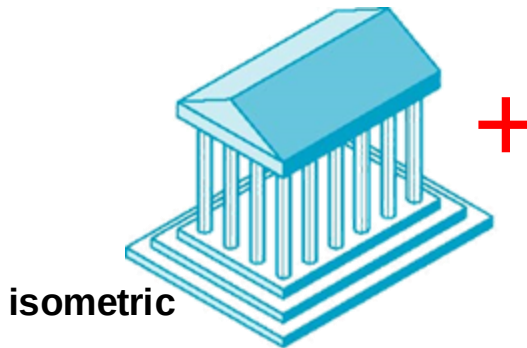
- The word *orthographic* refers to the projection system that is used to derive multiview drawings based on the glass box model.
- Drawings that appear on a surface are the view a person sees on the transparent viewing plane that is positioned perpendicular to the viewer's line of sight and the object.
- In the orthographic system, the object is placed in a series of positions (plan or elevation) relative to the viewing plane.



Multiview Orthographic Projection

- Projection plane parallel to principal face
- Usually from front, top, side views

+ shown with isometric view (not one of orthographic views) in CAD/architecture so that 3D shape can be inferred



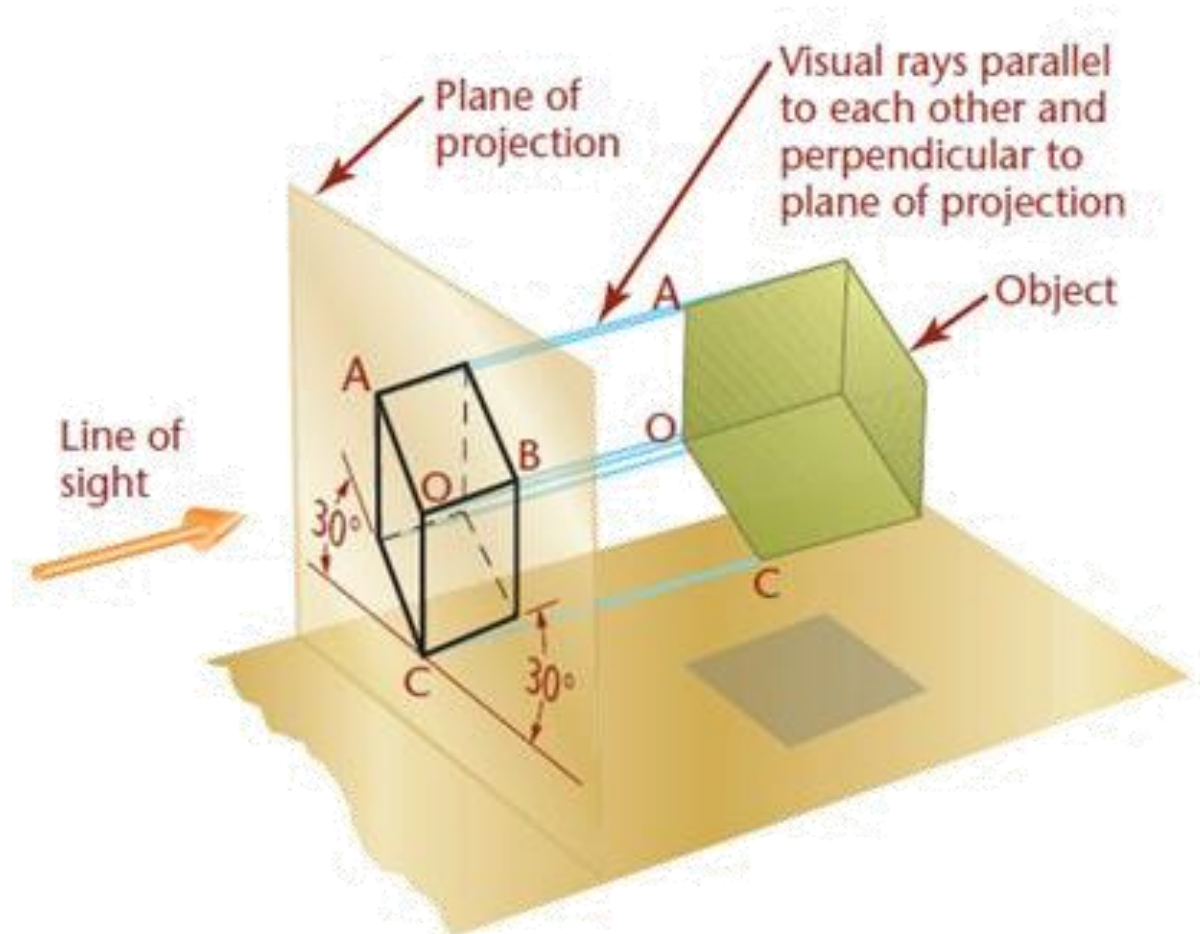
Advantages and Disadvantages of Orthographic Projection

- Preserves **both distances and angles**
 1. Shapes preserved
 2. Relative sizes preserved
 3. Can be used for measurements
 - Building plans
 - Manuals
- Cannot see what object really looks like because many surfaces are hidden from view
 - >> Therefore we often add the isometric view too

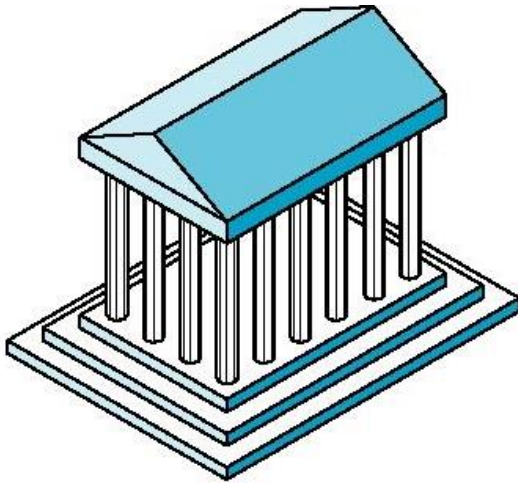
Axonometric Projections

Allow projection plane to move relative to object

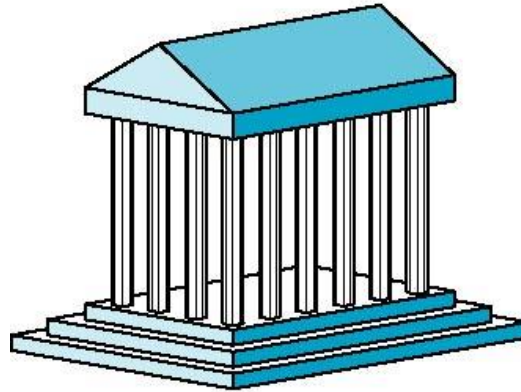
- **projectors are still orthogonal** to projection plane
- but projection plane can have any orientation with respect to the object



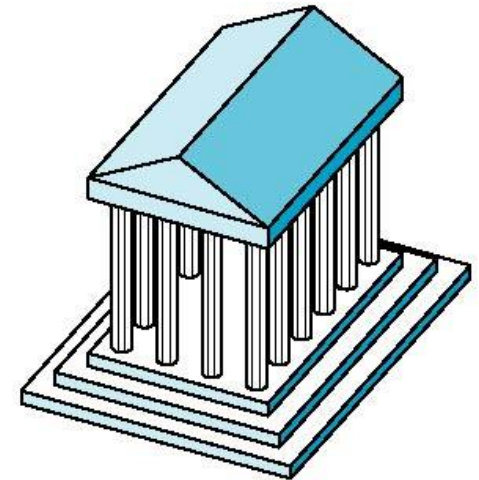
Types of Axonometric Projections



Isometric

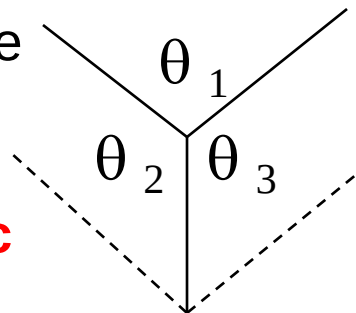


Dimetric



Trimetric

classified by how many angles of a corner of a projected cube are the same



- all 3 same = 120 : **isometric**
- 2 of them same : **dimetric**
- all different : **trimetric**

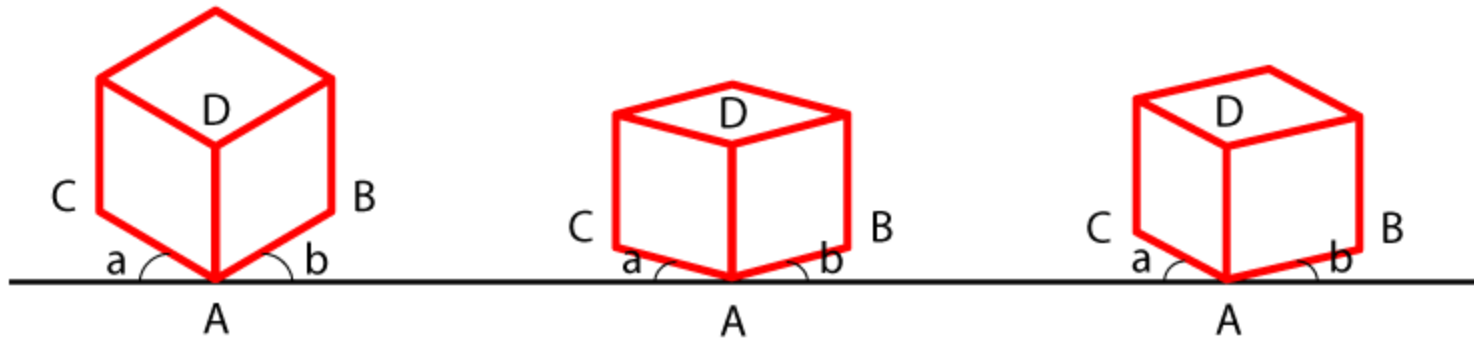
If the projection plane is placed symmetrically

-wrt to all 3 principal faces that meet at a corner of the rectangular object: isometric.

-wrt to 2 principal faces: dimetric.

-The general case is a trimetric view.

Types of Axonometric Projections



$$a = b = 30$$

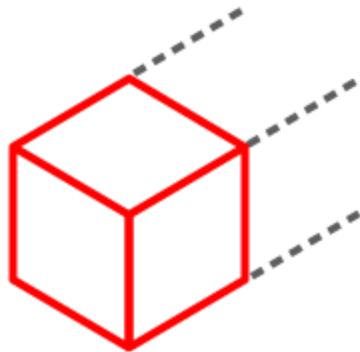
Isometric Projection

$$a = b$$

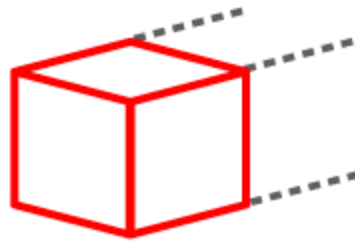
Dimetric Projection

$$a \neq b$$

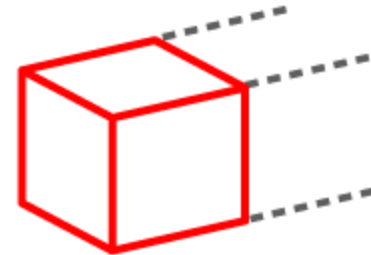
Trimetric Projection



Isometric Projection



Dimetric Projection



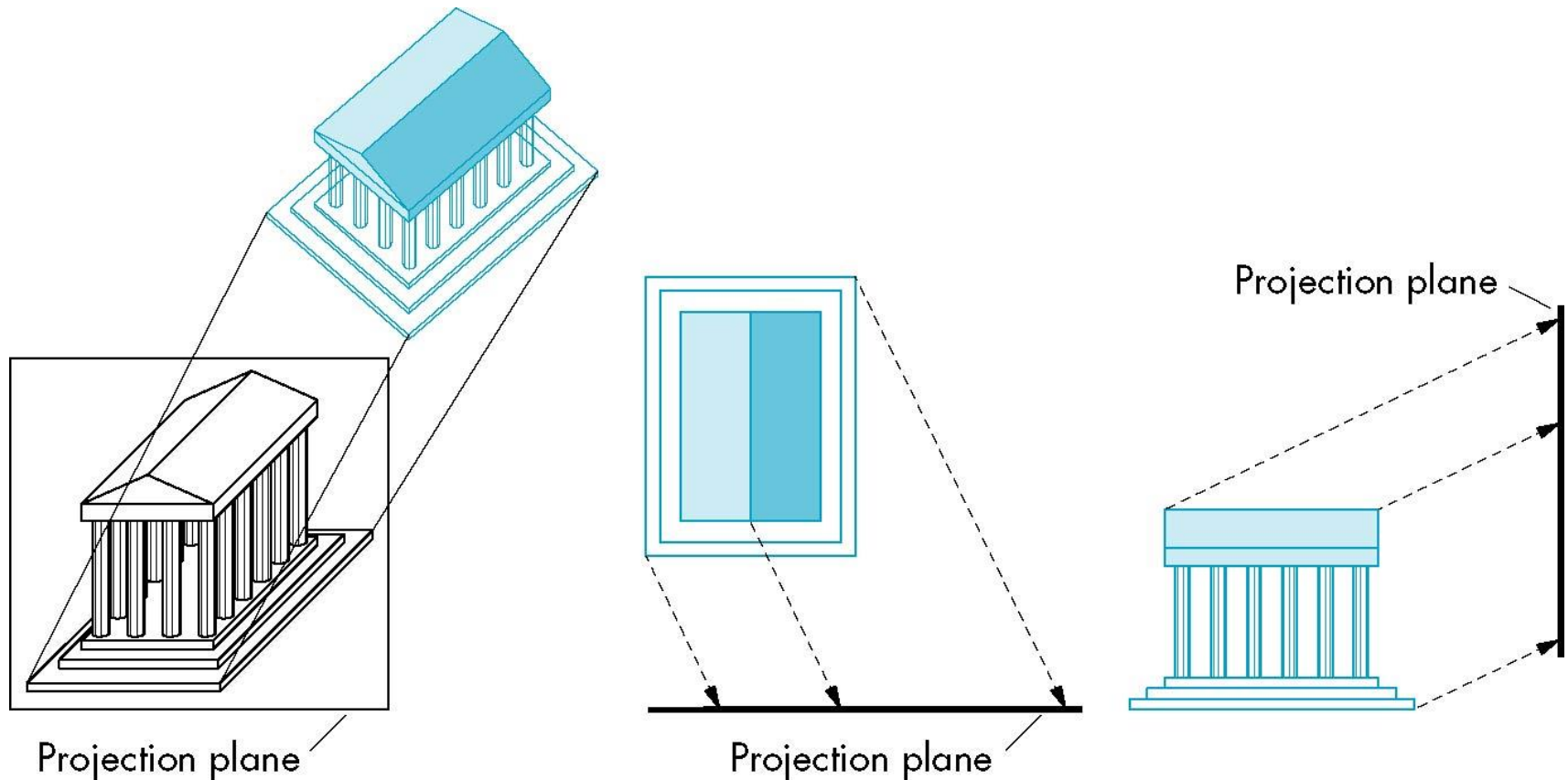
Trimetric Projection

Advantages and Disadvantages of Axonometric Projections

- Lines are scaled (foreshortened)
 - but can find scaling factors since the factors do not change with distance
- Angles are not preserved
 - Projection of a circle in a plane not parallel to the projection plane is an ellipse
- Can see three principal faces of a box-like object
- Some optical illusions possible
 - Parallel lines appear to diverge
- Does not look real because far objects are scaled the same as near objects
- Used in CAD applications

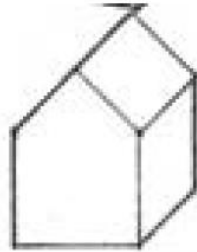
Oblique Projection

Arbitrary relationship between projectors and projection plane

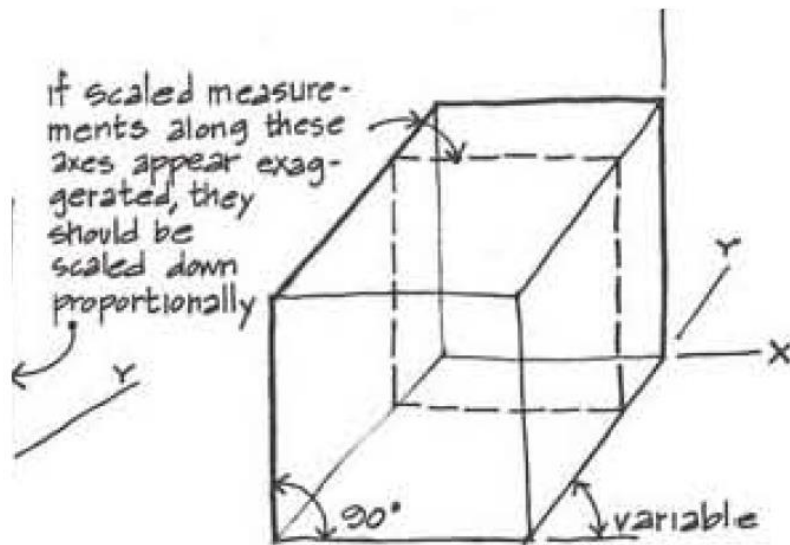


Elevation Oblique

Principal vertical face of rectangular form is parallel with the picture plane.

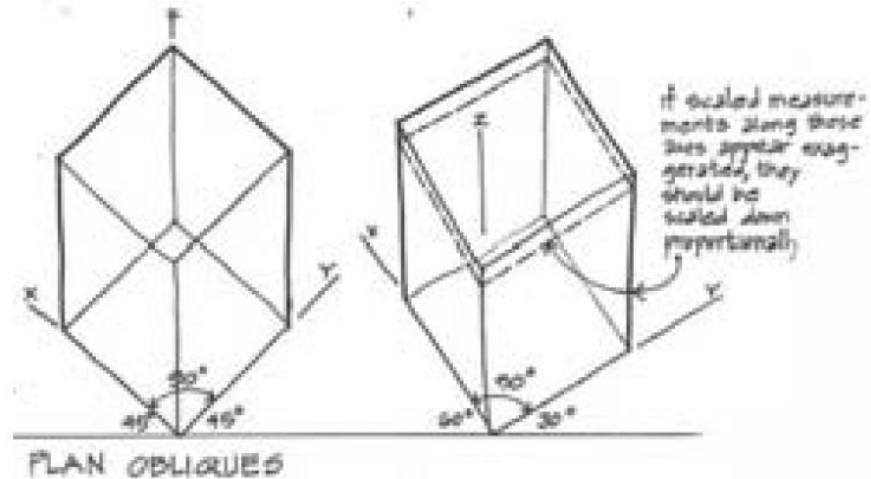
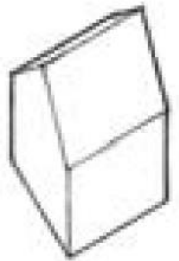


Source: Ching. Architectural graphics. 2003 page 115



Plan Oblique

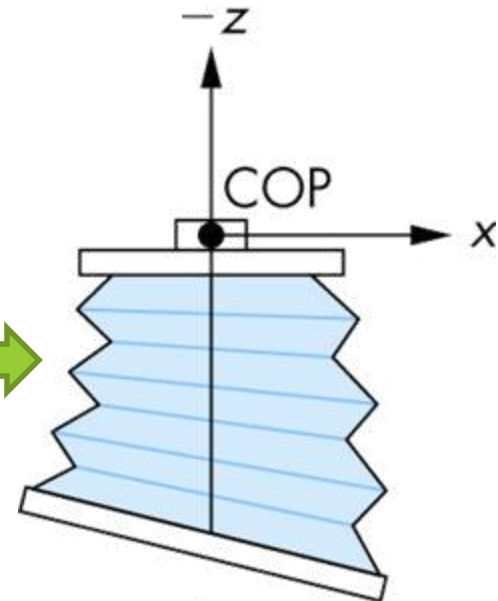
Principal horizontal face of rectangular form is parallel with the picture plane.



- “Unnatural” as physical viewing devices (e.g., eye, most cameras) have lens that is in a fixed relationship with the image plane (parallel)
 - Except with Bellows Cameras

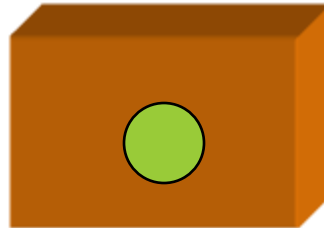


- No problem to program it, though!



Advantages and Disadvantages of Oblique Projections

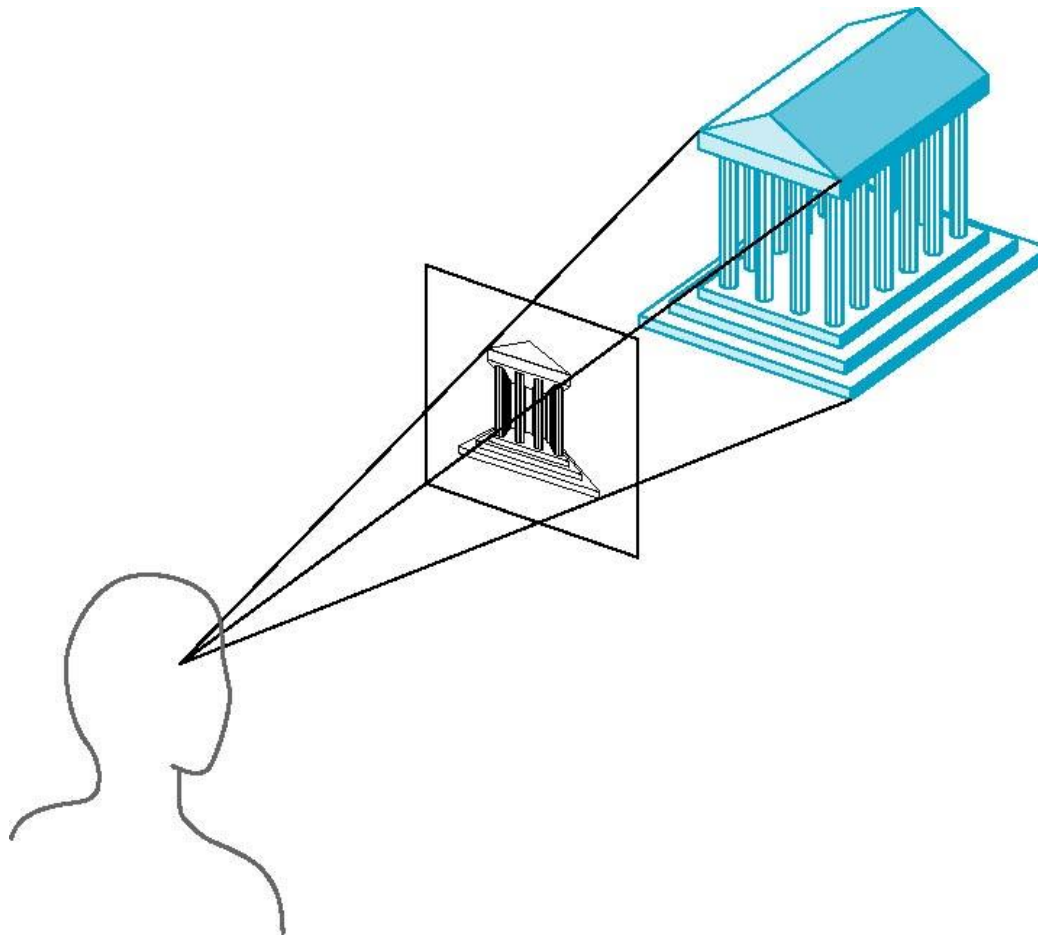
- Can pick the angles to emphasize a particular face
 - Architecture: plan oblique, elevation oblique
- Angles in faces parallel to projection plane are preserved while we can still see “around” side



- In physical world, cannot create with simple camera; possible with bellows camera or special lenses (architectural)

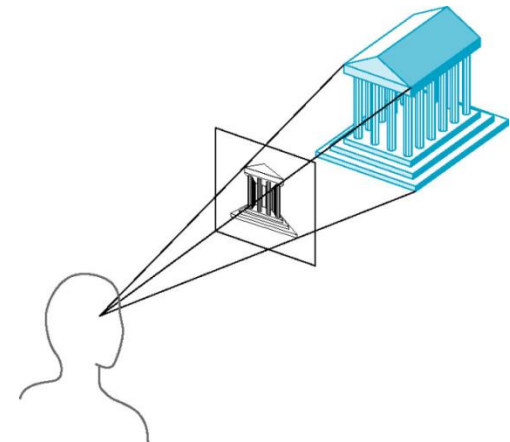
Perspective Projection

Projectors converge at center of projection



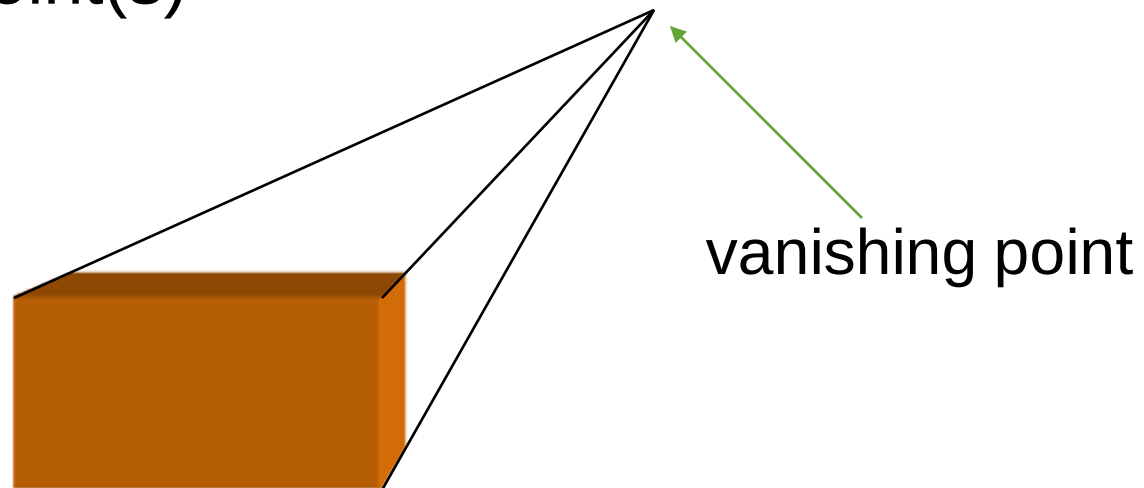
Perspective Viewing

- characterized by diminution of size
 - Projectors are not parallel
 - Leads to natural appearance
 - Amount by which line is foreshortened depends on how far from user
- In classical perspective views, viewer is located symmetrically wrt projection plane
 - Viewing pyramid (frustum) determined by window in projection plane and COP is a symmetric or right pyramid (apex directly above the centre of the base)
- One-, two-, or three-point perspectives
 - How many of the three principal directions in object are parallel to the projection plane
 - How many vanishing points for projections



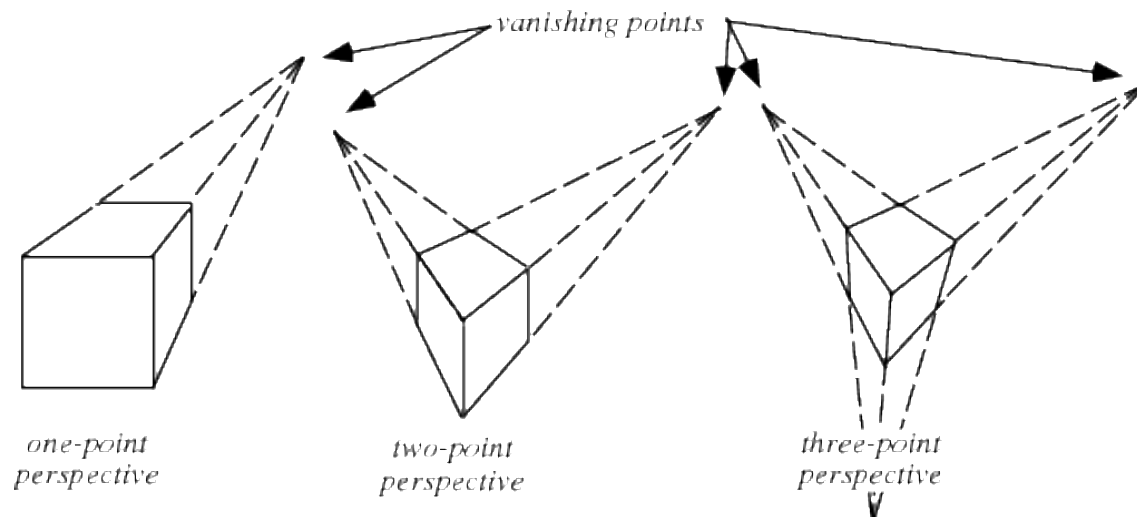
Vanishing Points

- Parallel lines on the object that are not parallel to the projection plane converge at a single point in the projection (the *vanishing point*)
- Drawing simple perspectives by hand uses these vanishing point(s)



Vanishing Points

- Parallel lines on the object that are not parallel to the projection plane converge at a single point in the projection (the *vanishing point*)
- Drawing simple perspectives by hand uses these vanishing point(s)



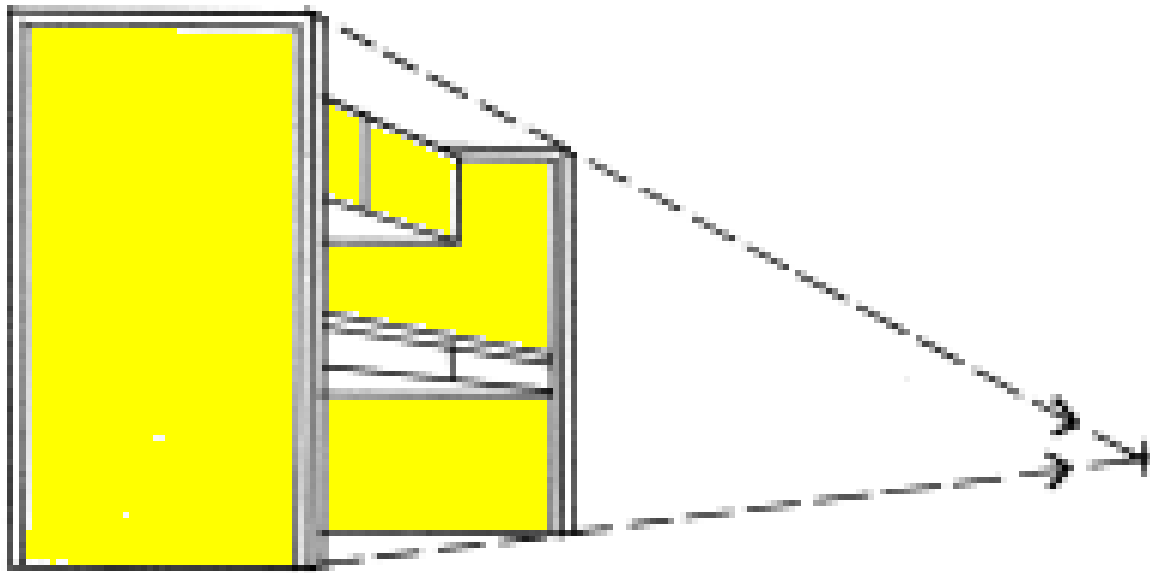
One-Point Perspective

- One principal face parallel to projection plane
- One vanishing point for cube



One-Point Perspective

- receding lines or sides of an object appear to vanish to a single point on the horizon



ONE POINT

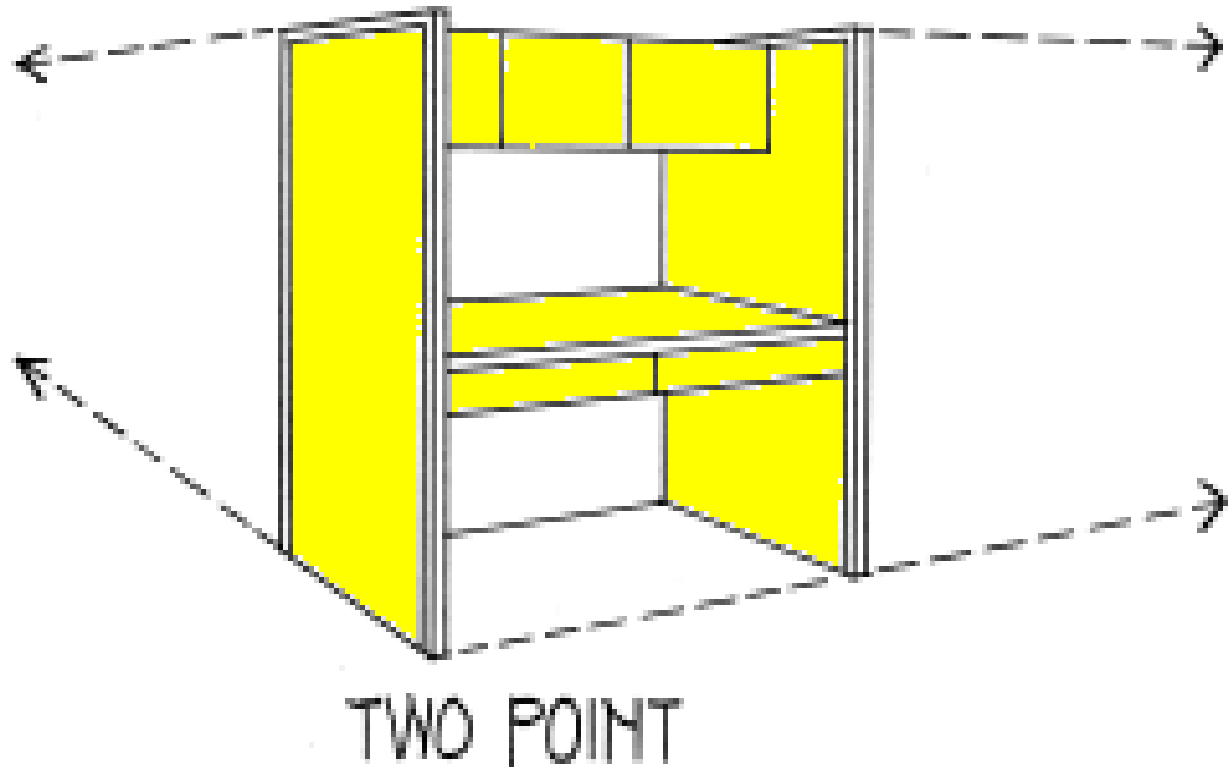
Two-Point Perspective

- One principal direction parallel to projection plane
- Two vanishing points for cube



Two-Point Perspective

- The two-point perspective is one of the most widely used of the three types, as it **portrays the most realistic view for the observer**



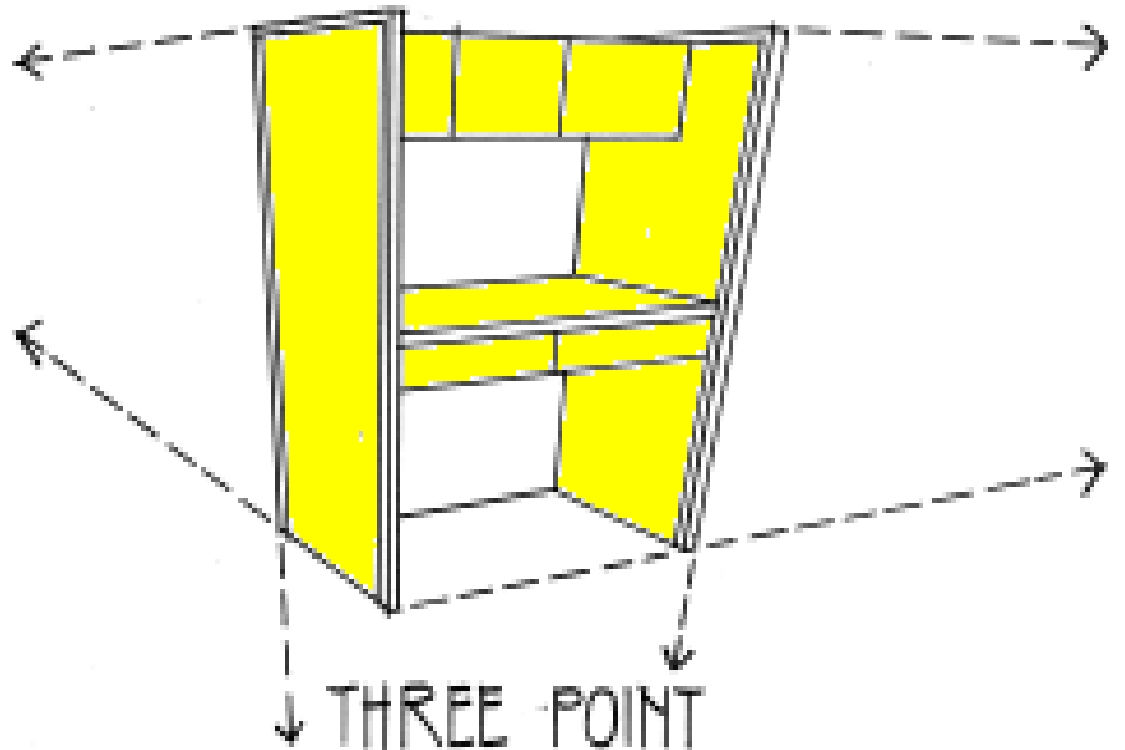
Three-Point Perspective

- No principal face or direction parallel to projection plane
- Three vanishing points for cube



Three-Point Perspective

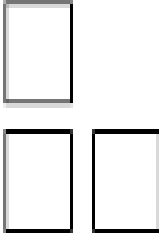
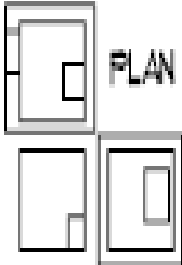
- Generally drawn with the viewer at a distance above the horizon (**bird's-eye view**) or below the horizon (**worm's-eye view**).
- Used mostly for very tall buildings and is rarely used in interior spaces, unless they are multistoried.
- More complicated than the former two types, as a third vanishing point is introduced, which precludes all parallel lines.

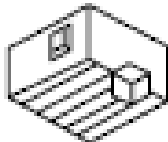
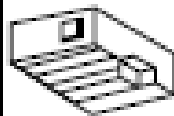
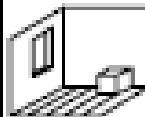
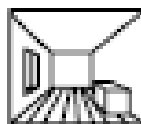
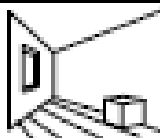


Advantages and Disadvantages of Perspective Projection

- Objects further from viewer are projected smaller than the same sized objects closer to the viewer (***diminution***)
>> Looks realistic
- Equal distances along a line are not projected into equal distances (***nonuniform foreshortening***)
- Angles preserved only in planes parallel to the projection plane
- More difficult to draw by hand than parallel projections, but not difficult by computer

CLASSIFICATION OF DRAWING SYSTEMS

MULTIVIEW	TYPE			APPLICATION		RELATIONSHIP OF OBJECTS TO PICTURE PLANE
				OBJECT	INTERIORS	
		ORTHOGRAPHIC		 PLAN/ELEVATION	 ELEVAT'N SECT'N	AN OBJECT'S RECTANGULAR FACES ARE PARALLEL TO THE PICTURE PLANE.

SINGLEVIEW				
PARALLEL LINES REMAIN PARALLEL TO EACH OTHER	PARALLEL LINES	AXONOMETRIC	 ISOMETRIC	 THE THREE PRINCIPAL AXES MAKE EQUAL ANGLES (30°) WITH THE PICTURE PLANE. ALL LENGTHS ARE EQUAL.
			 DIMETRIC	 THE TWO PRINCIPAL AXES MAKE EQUAL ANGLES WITH THE PICTURE PLANE, AND TWO LENGTHS ARE EQUAL. OBJECTS CAN BE ROTATED AT VARIOUS ANGLES.
			 TRIMETRIC	 EACH OF THE TWO PRINCIPAL AXES MAKES A DIFFERENT ANGLE WITH THE PICTURE PLANE. HEIGHT IS REDUCED, SIMILAR TO A DIMETRIC.
	OBLIQUE	 ELEVATION	 THE FACE (ELEVATION) OF THE OBJECT IS PARALLEL TO THE PICTURE PLANE. DEPTHS ARE USUALLY REDUCED IN RATIO.	
		 PLAN	 THE TOP VIEW (OR PLAN) OF THE OBJECT IS PARALLEL TO THE PICTURE PLANE. HEIGHTS ARE USUALLY REDUCED.	
	PERSPECTIVE PARALLEL LINES APPEAR TO CONVERGE TO VANISHING POINTS	 ONE-POINT	 ONE FACE IS PARALLEL TO THE PICTURE PLANE. PROJECTOR LINES CONVERGE TO ONE POINT.	
		 TWO-POINT	 VERTICAL FACES ARE AT AN ANGLE TO THE PICTURE PLANE. PROJECTOR LINES CONVERGE TO TWO POINTS.	
		 THREE-POINT	 VERTICAL FACES ARE AT AN ANGLE TO THE PICTURE PLANE. PROJECTOR LINES CONVERGE TO THREE POINTS.	

COMPUTER VIEWING

Lecturer: Dr. Ufuk Çelikcan

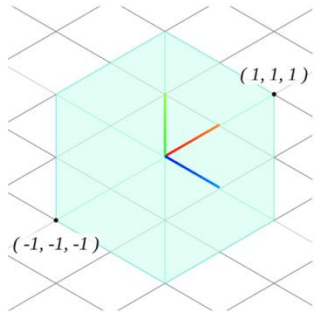
Based on the slides by: E. Angel and D. Shreiner

Objectives

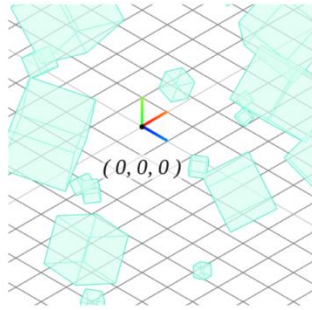
- Introduce the mathematics of projection
- Introduce WebGL viewing functions
- Look at alternate viewing javascript libraries

Computer Viewing

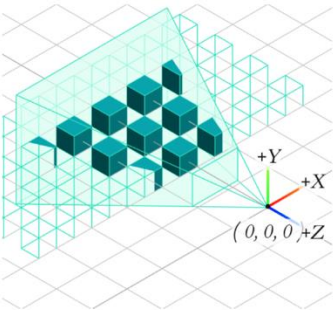
- There are 3 aspects of the viewing process, all of which are implemented in the pipeline
 - **Positioning the camera:**
 - Setting the model-view matrix
 - **Selecting a lens:**
 - Setting the projection matrix
 - **Clipping:**
 - Setting the view volume



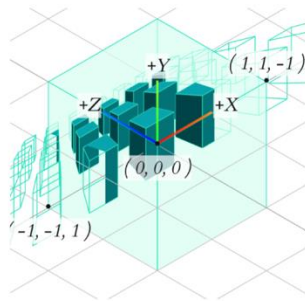
(A) Model Space



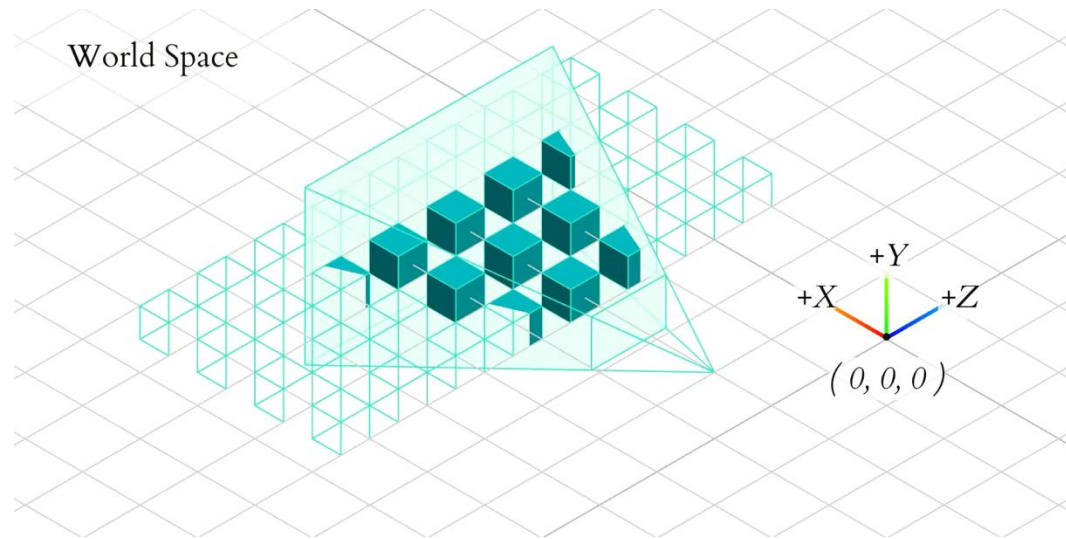
(B) World Space



(C) Camera Space



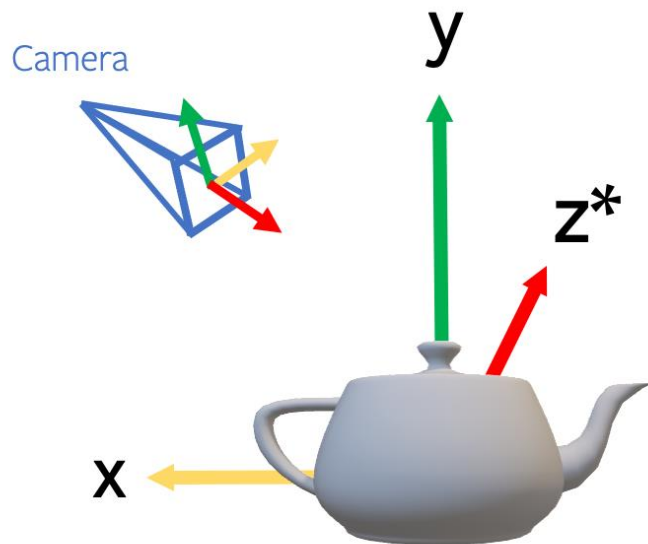
(D) Perspective Projection



World Frame and Camera Frame

- When we work with representations, we work with **n-tuples** which are arrays of n scalars
 - In 3D graphics, we use 4-tuples
- >> Changes in frames are defined by 4x4 matrices
- In WebGL, the base frame that we start with is **the world frame**
 - Then we represent entities in **the camera frame** by changing the world representation using the model-view (MV) matrix
 - Initially these frames are the same ($\mathbf{MV}=\mathbf{I}$) until we change them using the model-view matrix

World space



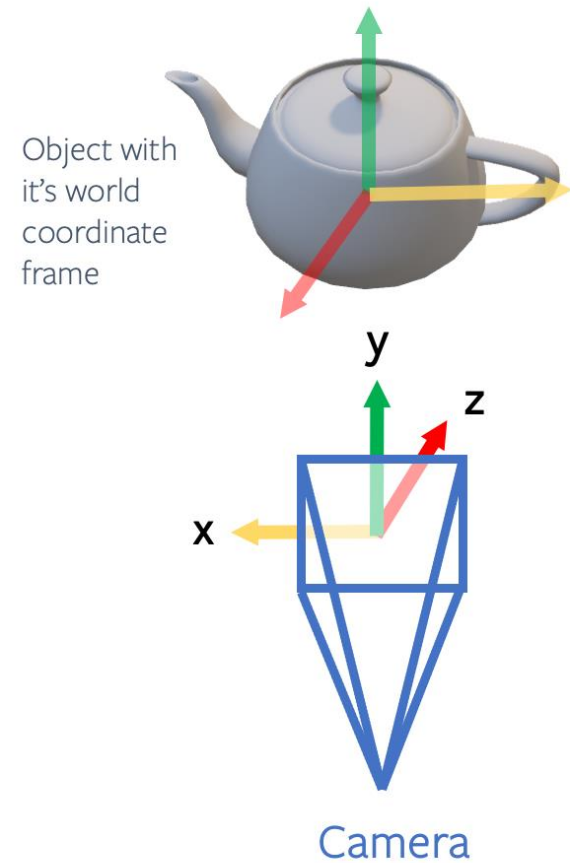
World-to-camera
transform



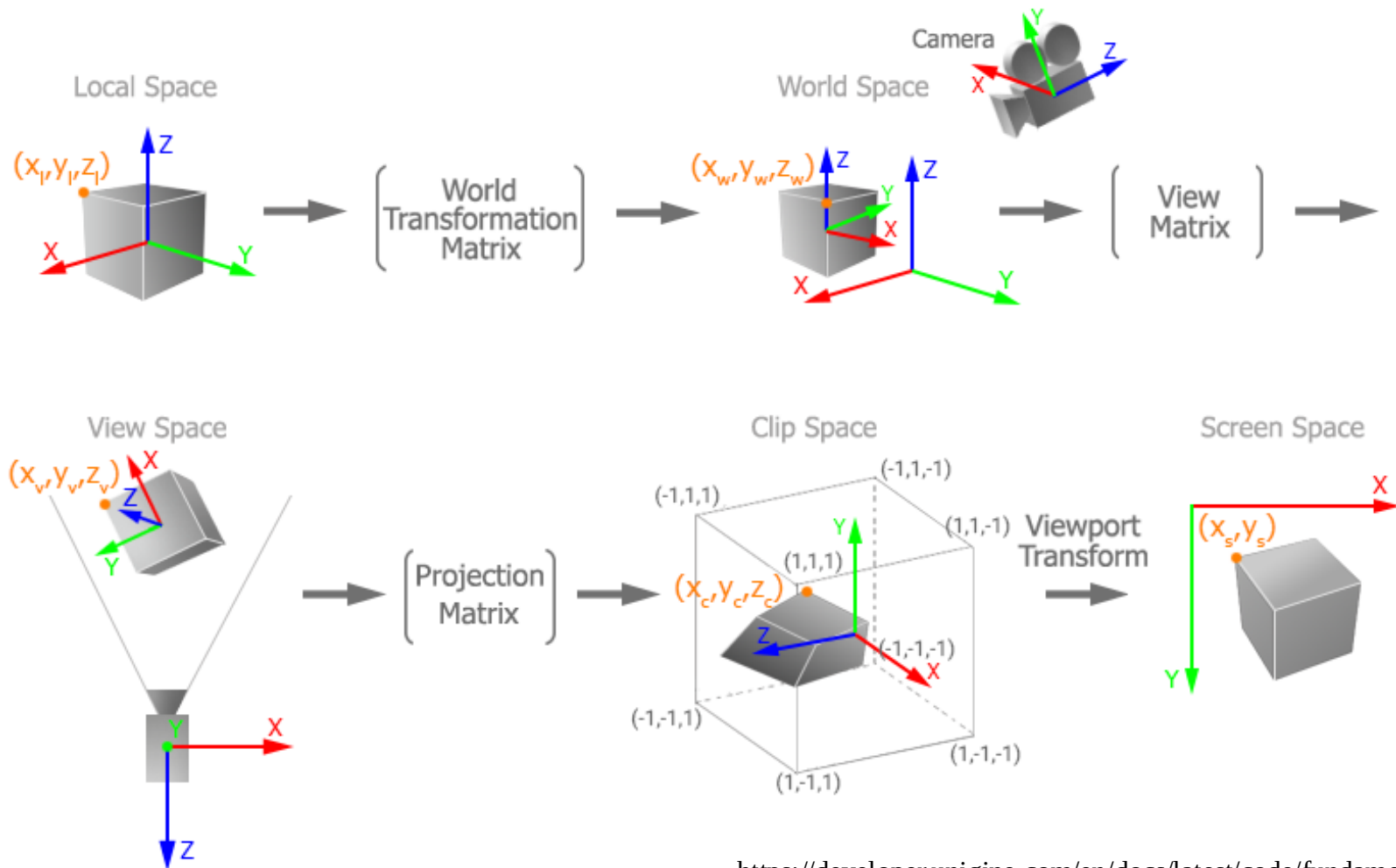
Camera-to-world
transform

View space

(The view from the camera)



- **Local space** is where all vertices of an object are defined relative to the center of that object.
- **World space** is where all vertices of an object are defined relative to the center of the world.
- **View space** is where all vertices of an object are defined relative to the camera.
- **(Normalized) Clip space** is where all vertices of an object are defined in a cuboid of $[-1;1]$ dimensions for every axis. Here, Z coordinate of the vertex specifies its depth (how far away the vertex is from the screen).
- **Screen space** is where all vertices of an object are flattened and have screen coordinates.

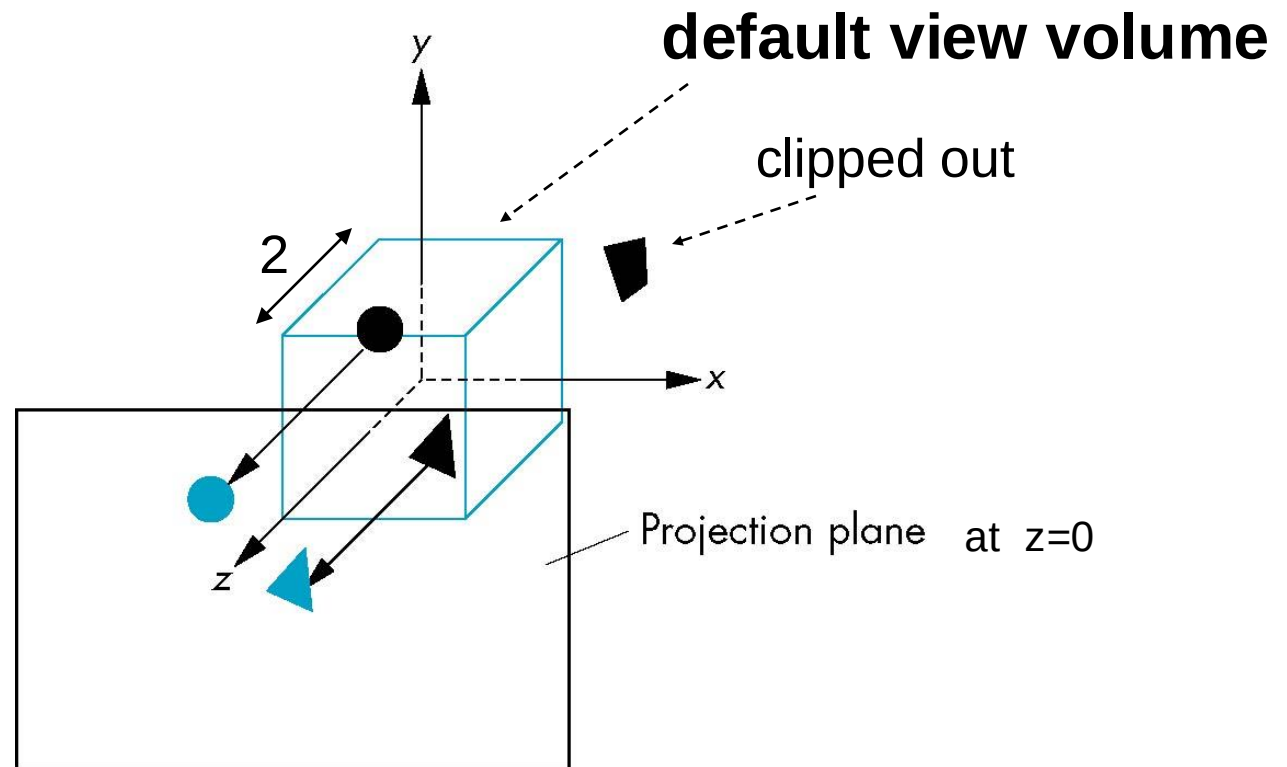


The WebGL Camera

- In WebGL, initially (by default) the object and camera frames are the same
 - In other words, the default model-view matrix is an identity (I) matrix
 - The camera is **located at origin** and **points in the negative z direction**
 - WebGL also specifies a **default view volume** that is **a cube with sides of length 2 centered at the origin**
 - Same as NDC cube so no projection needed at first until you apply a model-view transform

Default Projection

Default projection is orthogonal

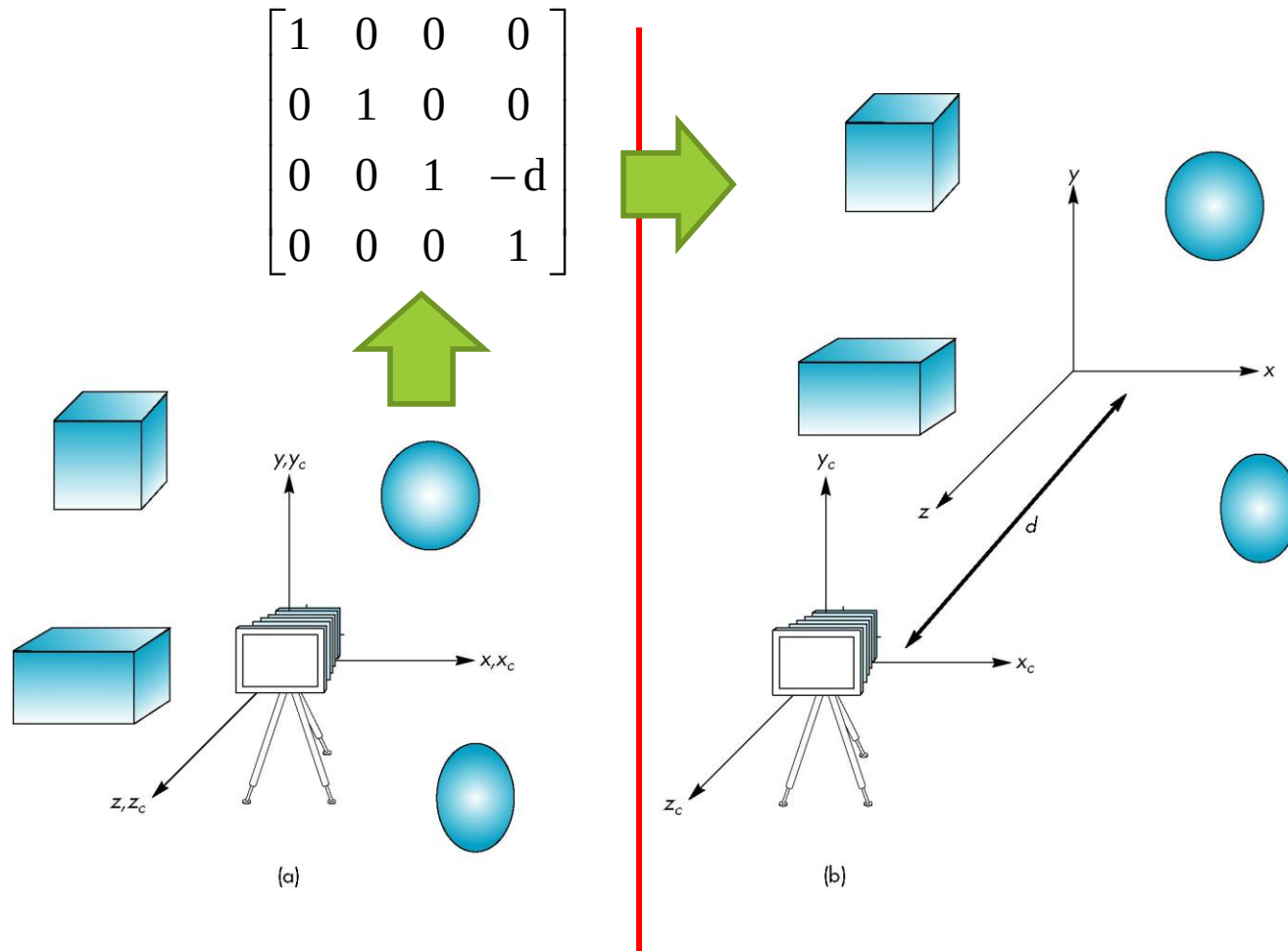


Moving the Camera Frame

- If we want to visualize object with both positive and negative z values we can either
 - Make a **view matrix**: to move the camera in the positive z direction
 - >> Translates the camera frame
 - Make a **world matrix**: to move the objects in the negative z direction
 - >> Translates the world frame
- **Both of these views are equivalent and are determined by the same model-view matrix**
 - Want a translation: `translate(0.0, 0.0, -d)`
 - Where $d > 0$

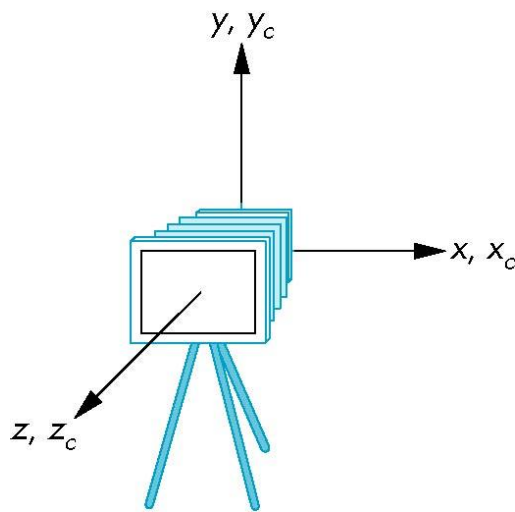
Moving the Camera

If objects are on both sides of $z=0$, we must move the camera frame

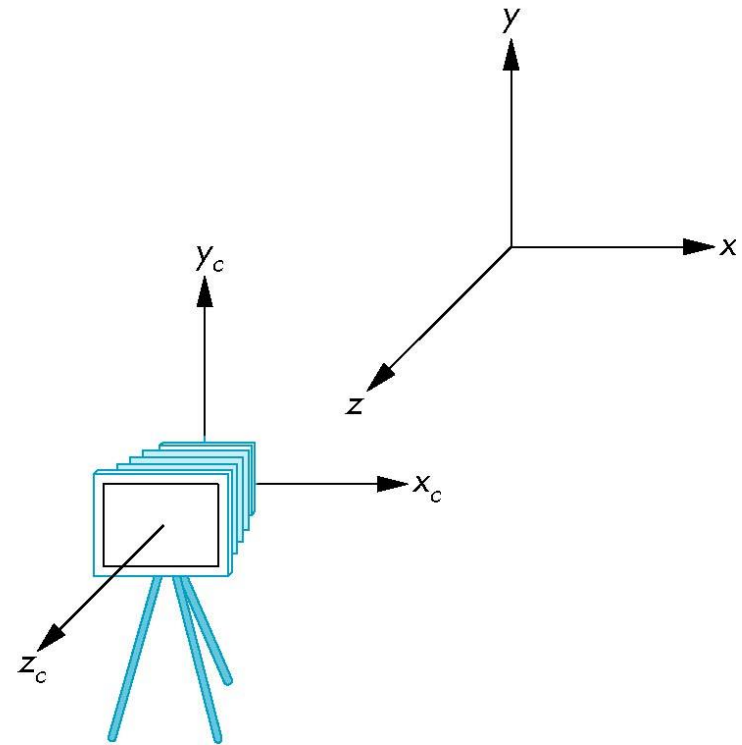


Moving Camera back from Origin

frames after translation by $-d$
 $d > 0$



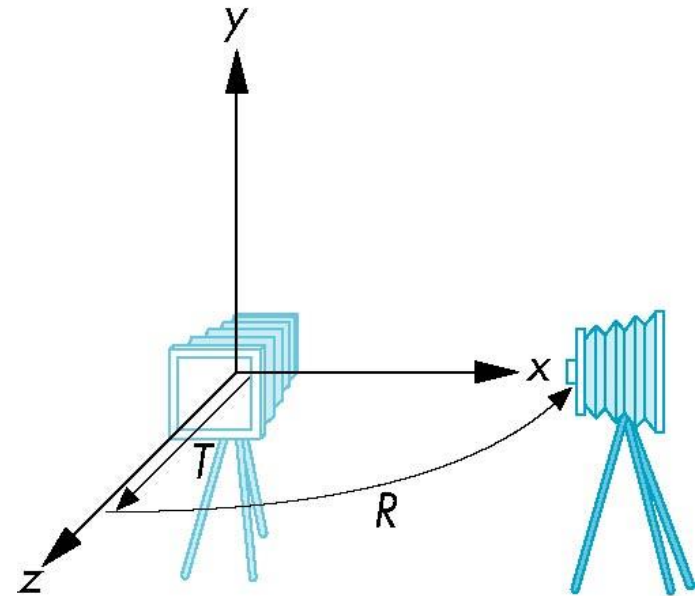
(a)



(b)

Moving the Camera

- We can move the camera to any desired position by a sequence of rotations and translations
- Example: side view
 - Move it away from origin: T
 - Rotate the camera about world origin: R
 - Model-view matrix $M = RT$so that $Mv = RTv$



The lookAt Function

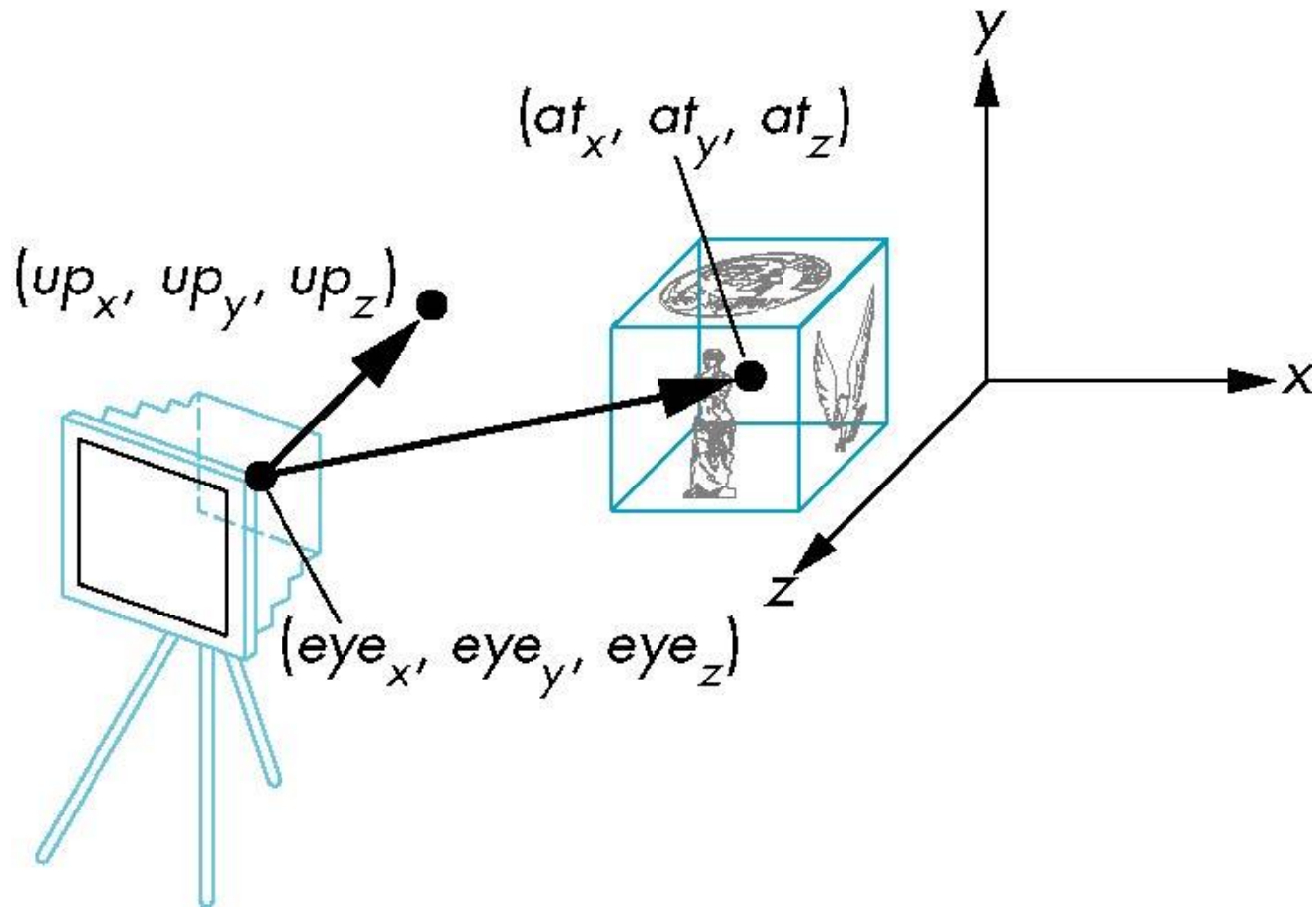
- The MV library contains the *lookAt* function to form the required modelview matrix through a simple interface
- Or you can write your own view function
 - Can concatenate with modeling transformations

```
mat4 mv = lookAt(vec4 eye, vec4 at, vec4 up) ;
```

- Note the need for specifying an **up direction**

lookAt

`lookAt(eye, at, up)`



Other Viewing APIs

- The lookAt function is only one possible API for positioning the camera
- Others include
 - View reference point, view plane normal, view up (PHIGS, GKS-3D)
 - Yaw, pitch, roll
 - Elevation, azimuth, twist
 - Direction angles

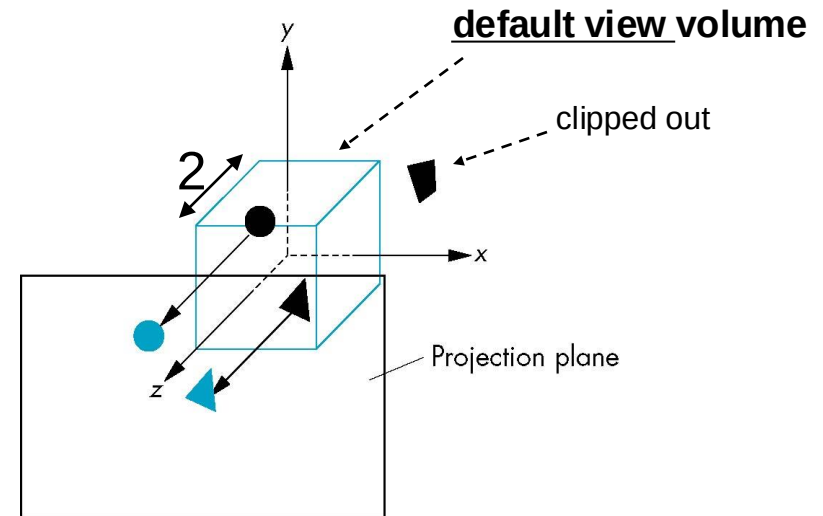
Projections and Normalization

- The default projection in the eye (camera) frame is orthogonal
- For points within the default view volume

$$\mathbf{X}_p = \mathbf{X}$$

$$\mathbf{y}_p = \mathbf{y}$$

$$\mathbf{z}_p = 0$$



- Most graphics systems use ***view normalization***
 - All other views are converted to the default view by transformations that determine the projection matrix
 - Allows the use of the same pipeline for all views

Homogeneous Coordinate Representation

default orthographic projection:

$$x_p = x$$

$$y_p = y$$

$$z_p = 0$$

$$w_p = 1$$

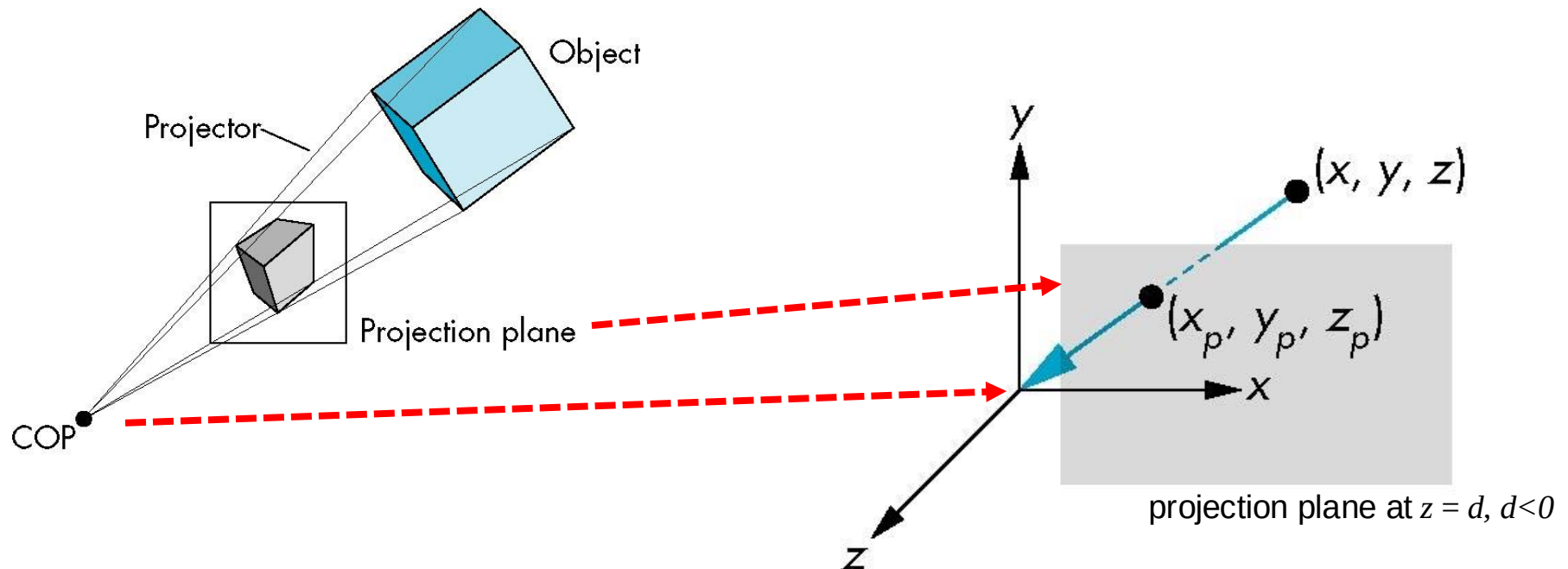
$$\mathbf{p}_p = \mathbf{M}\mathbf{p}$$

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

In practice, we can let $\mathbf{M} = \mathbf{I}$ and set the z term to zero later

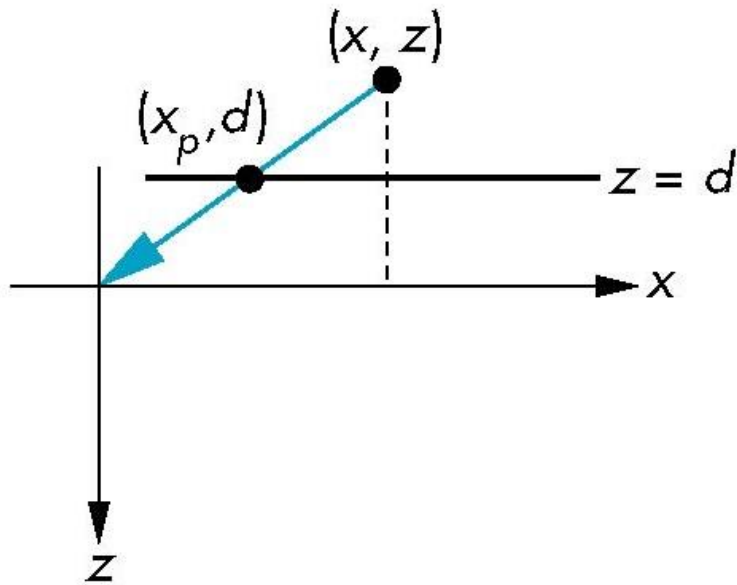
Simple Perspective

- Center of projection at the origin
- Projection plane at $z = d$, where $d < 0$



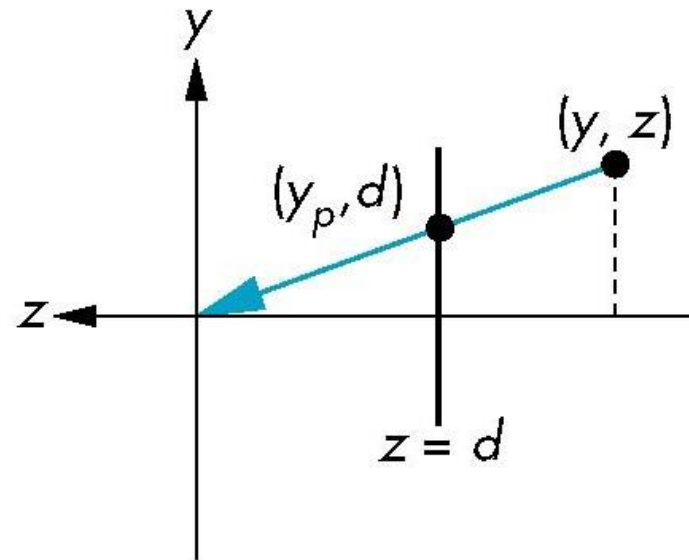
Perspective Equations

Consider top and side views



$$x_p = \frac{x}{z/d}$$

$$y_p = \frac{y}{z/d}$$



$$z_p = d$$

Homogeneous Coordinate Form

consider $\mathbf{q} = \mathbf{M}\mathbf{p}$ where $\mathbf{M} =$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{bmatrix}$$

$$\Rightarrow \text{for } \mathbf{p} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \Rightarrow \mathbf{q} = \begin{bmatrix} x \\ y \\ z \\ z/d \end{bmatrix}$$

Perspective Division

- To make $w = 1$, **we must divide by w**
 - in this case $w = z/d$

>> This is called ***perspective division*** and it yields:

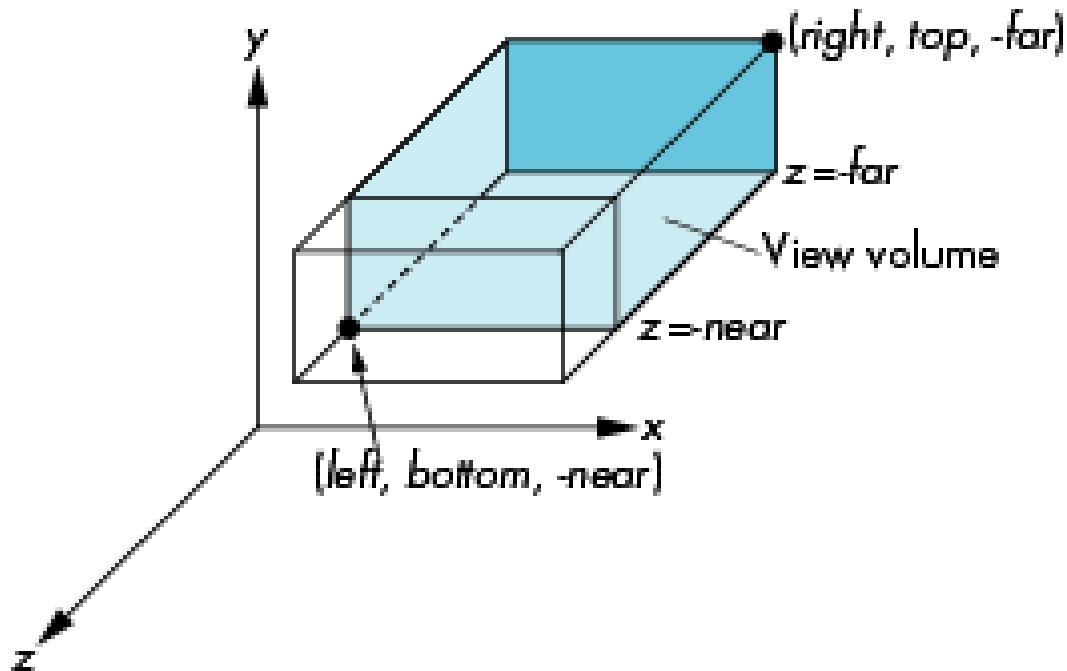
$$x_p = \frac{x}{z/d} \quad y_p = \frac{y}{z/d} \quad z_p = d$$

the desired perspective equations.

- We will consider the corresponding clipping volume with MV.js functions

Orthogonal Viewing

`ortho(left, right, bottom, top, near, far)`

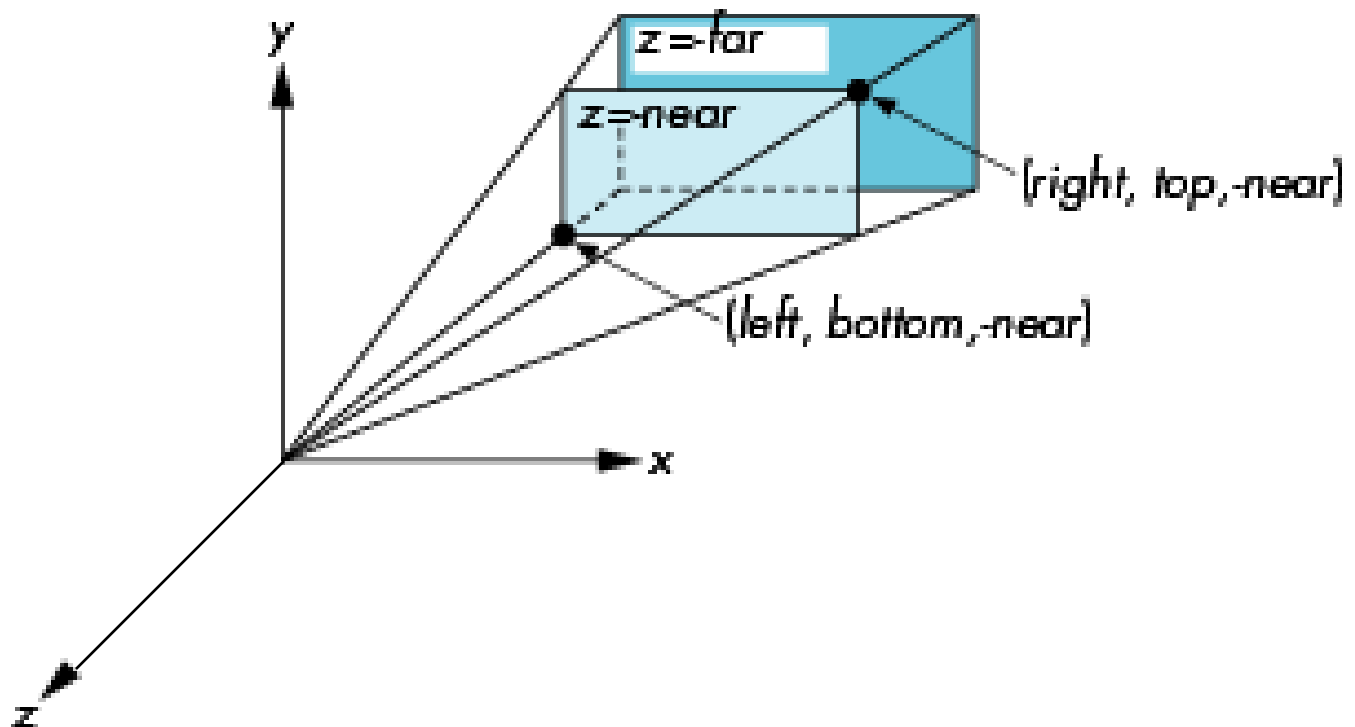


`near` and `far` are measured from camera

Perspective

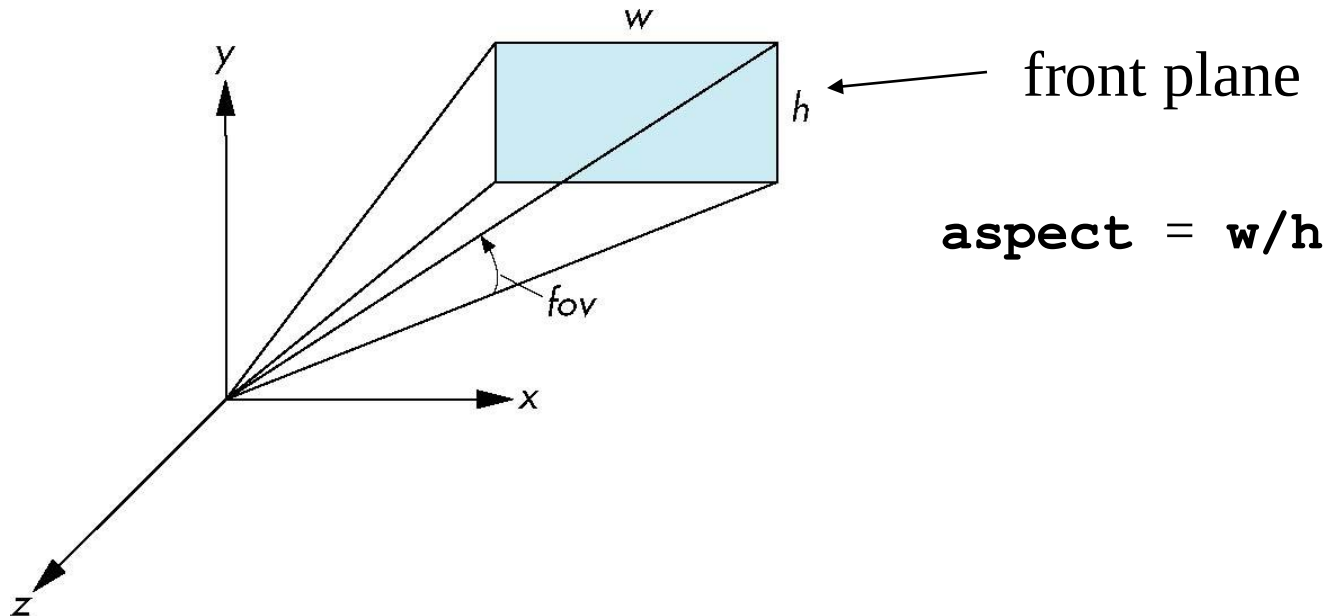
`frustum(left, right, bottom, top, near, far)`

*probably not available anymore



Using Field of View

- With **frustum** it is often difficult to get the desired view
- **perspective(fovy, aspect, near, far)** often provides a better way



under

<http://interactivecomputergraphics.com/8E/Code%20update/Common/>

- <http://interactivecomputergraphics.com/8E/Code%20update/Common/MVES6.js>
- <http://interactivecomputergraphics.com/8E/Code%20update/Common/MVnew.js>

PROJECTION MATRICES

Lecturer: Asst. Prof. Ufuk Çelikcan

Based on the slides by: E. Angel and D. Shreiner

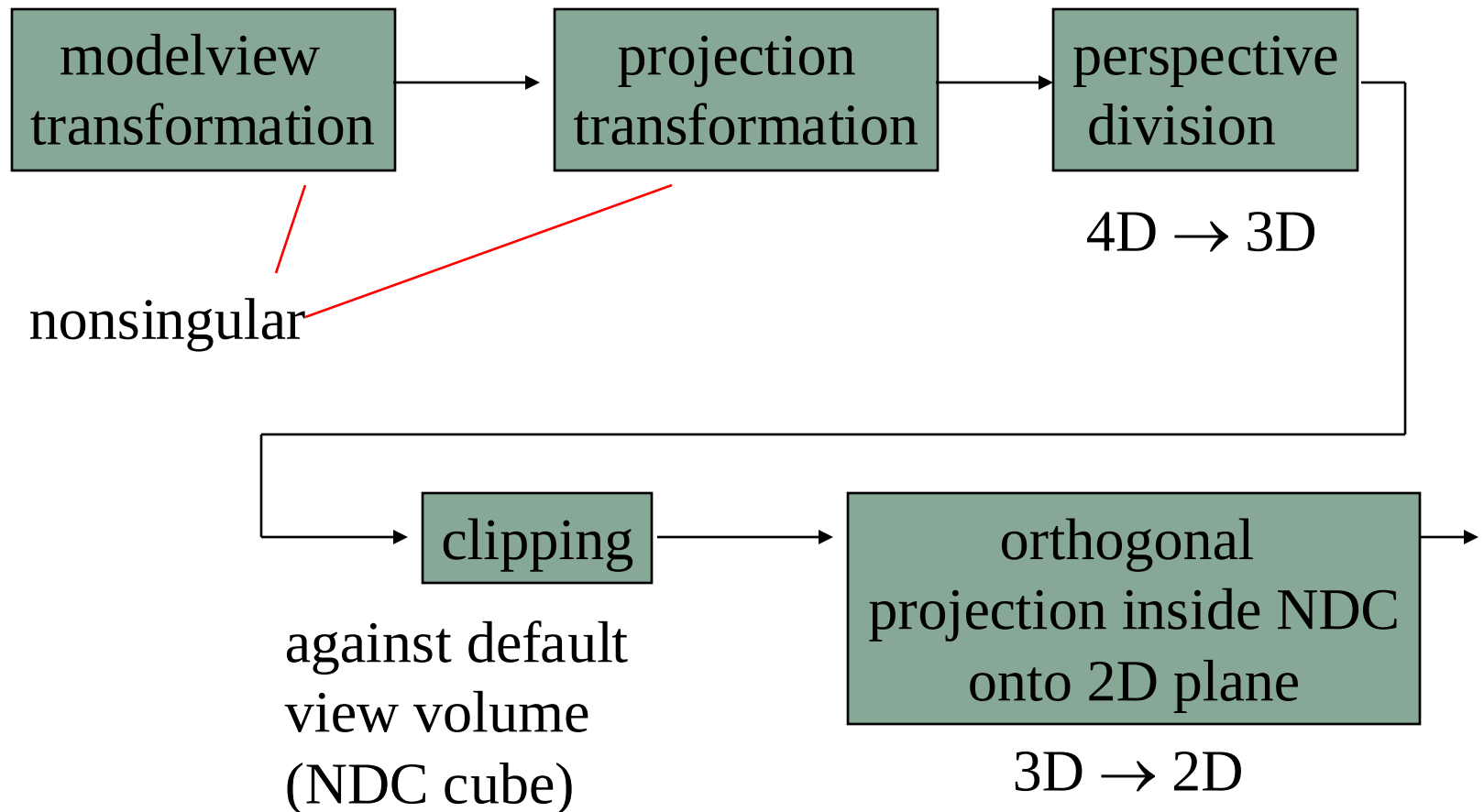
Objectives

- Derive the projection matrices used for standard WebGL projections
- Introduce oblique projections
- Introduce projection normalization

Normalization

- Rather than deriving a different projection matrix for each type of projection, we can do normalization at the end of the projection transformation stage so that we can simply use orthogonal projections within the default view volume.
- This way allows us to use standard transformations in the pipeline and makes for efficient clipping

Pipeline View



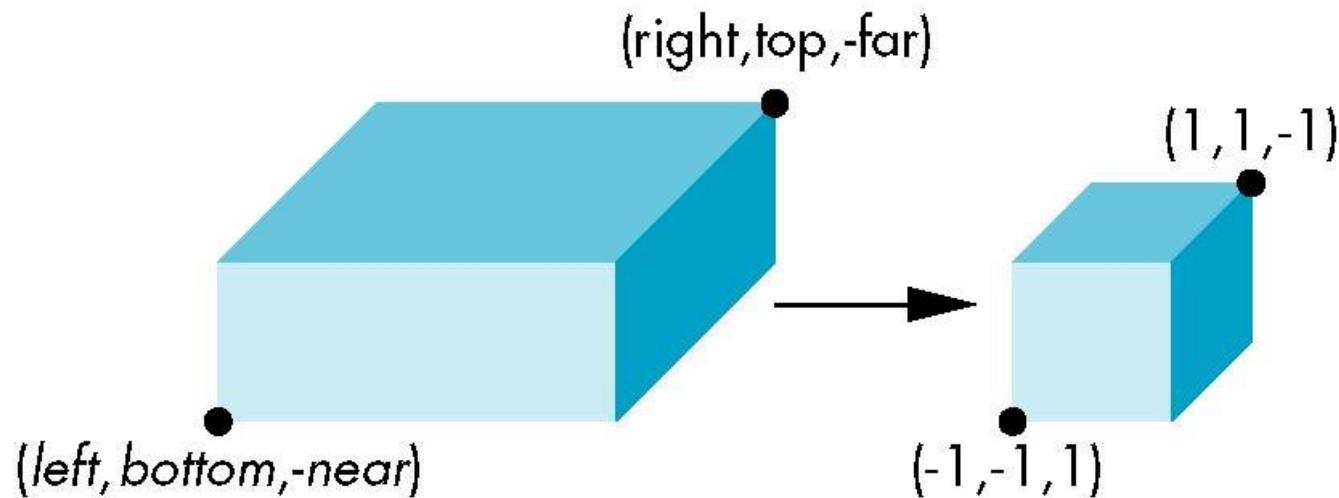
Notes

- We stay in four-dimensional homogeneous coordinates through both the model-view and projection transformations
 - Both these transformations are nonsingular
 - Default to identity matrices (orthogonal view)
- Normalization lets us clip against simple unit cube regardless of type of projection
- Delay final (3D $\rightarrow$ 2D) projection until end

Orthogonal Projection

`ortho(left, right, bottom, top, near, far)`

normalization $\Rightarrow$ find transformation to convert specified clipping volume to the default cube



Normalization Matrix

- 2 steps:

1. Move center to origin

$$T(-(right+left)/2 , -(top+ bottom)/2 , (far+ near)/2)$$

2. Scale to have sides of length 2

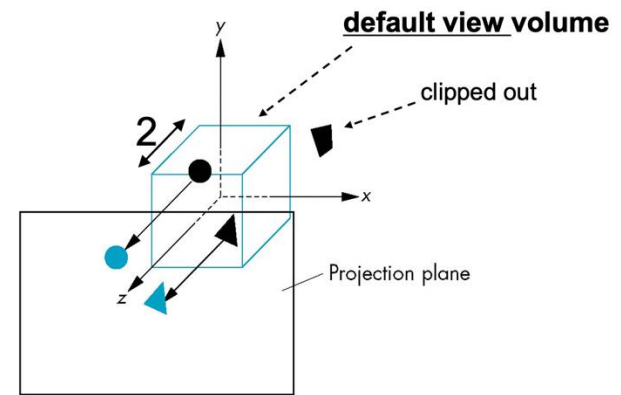
$$S(2/(right-left) , 2/(top-bottom) , 2/(far-near))$$

$$\mathbf{P} = \mathbf{ST} = \begin{bmatrix} \frac{2}{right-left} & 0 & 0 & -\frac{right+left}{right-left} \\ 0 & \frac{2}{top-bottom} & 0 & -\frac{top+bottom}{top-bottom} \\ 0 & 0 & \frac{-2}{far-near} & -\frac{far+near}{far-near} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Final Projection

- Set $z = 0$
- Equivalent to the homogeneous coordinate transformation

$$\mathbf{M}_{\text{orth}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



- Hence, general orthogonal projection in 4D is

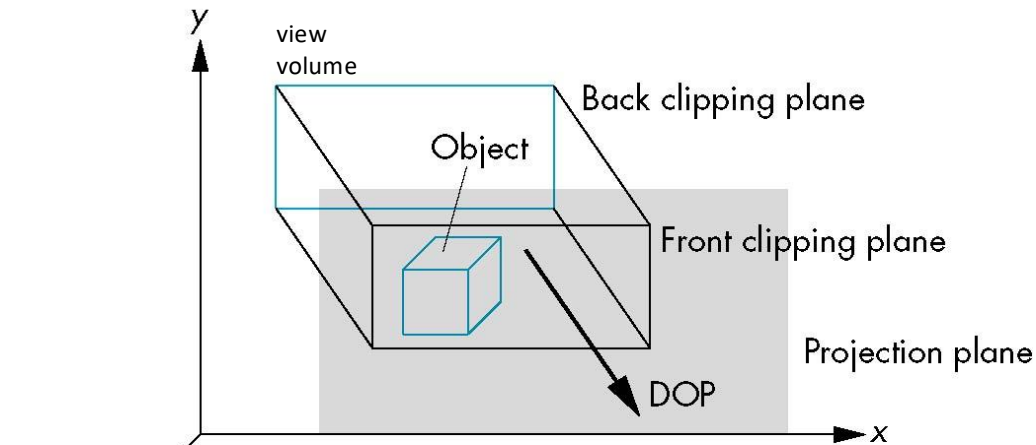
$$\mathbf{P} = \mathbf{M}_{\text{orth}} \mathbf{S} \mathbf{T}$$

Oblique Projections

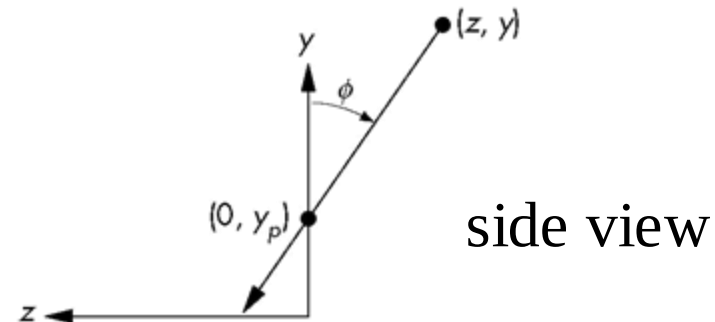
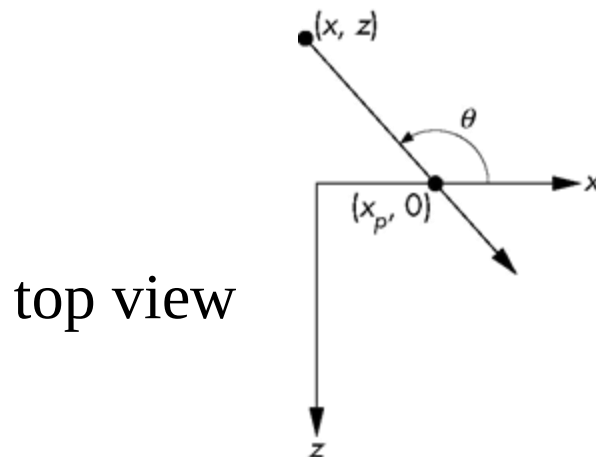


- if we look at the example of the cube, it appears as if the cube has been sheared
- Oblique Projection = **Shear + Orthogonal Projection**

General Shear



- the view volume for an oblique projection has the near and far clipping planes parallel to the projection plane
- the right, left, top, and bottom planes parallel to the DOP (direction of projection)



Shear Matrix

xy shear (z values unchanged)

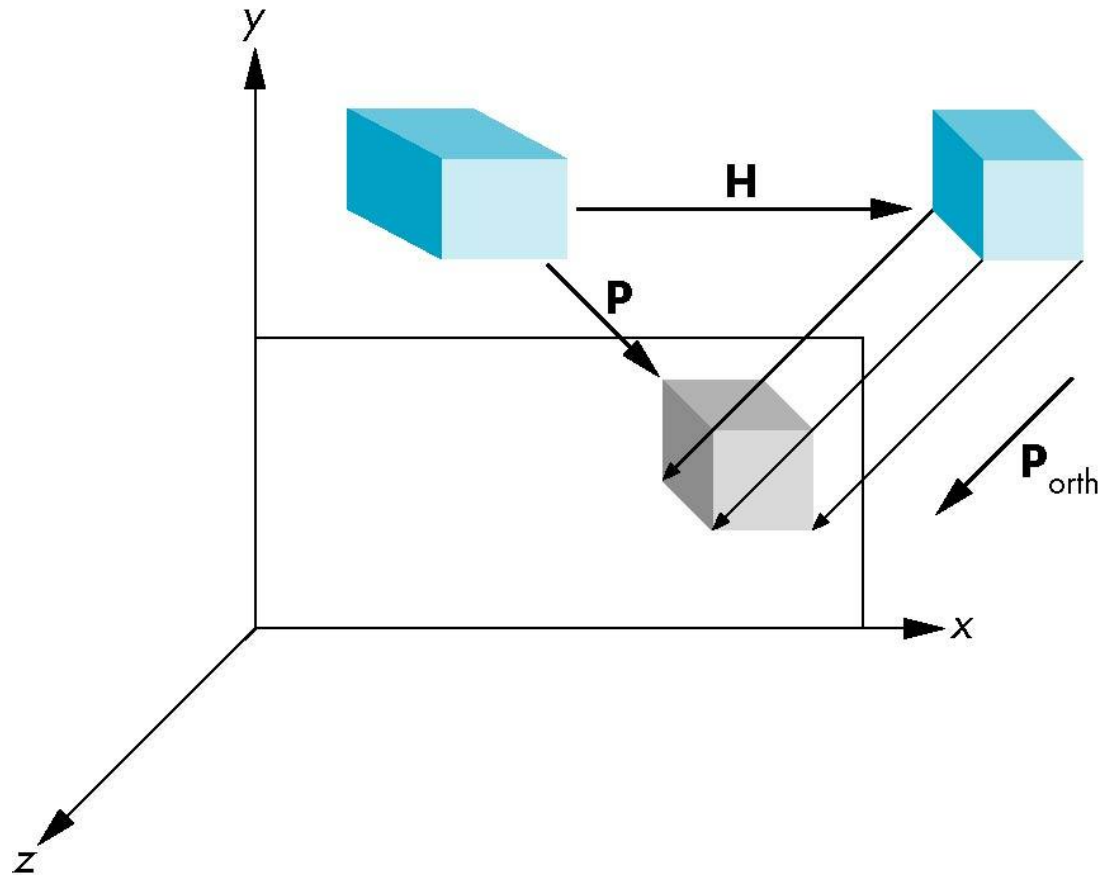
$$\mathbf{H}(\theta, \phi) = \begin{bmatrix} 1 & 0 & -\cot \theta & 0 \\ 0 & 1 & -\cot \phi & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Projection matrix

General case: $\mathbf{P} = \mathbf{M}_{\text{orth}} \mathbf{H}(\theta, \phi)$

with normalization: $\mathbf{P} = \mathbf{M}_{\text{orth}} \mathbf{S} \mathbf{T} \mathbf{H}(\theta, \phi)$

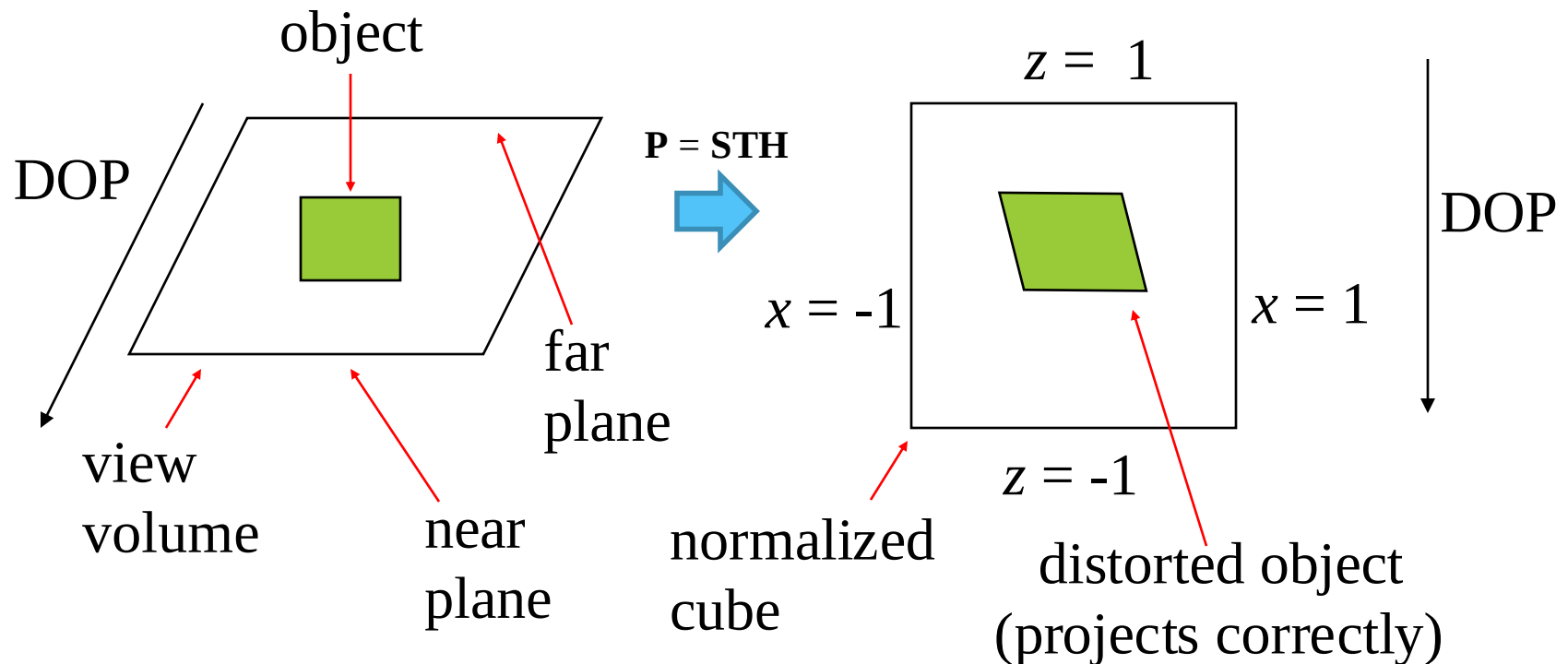
Equivalency



Effect on Clipping

- The projection matrix $\mathbf{P} = \mathbf{S}\mathbf{T}\mathbf{H}$ transforms the original view volume to the default clipping volume

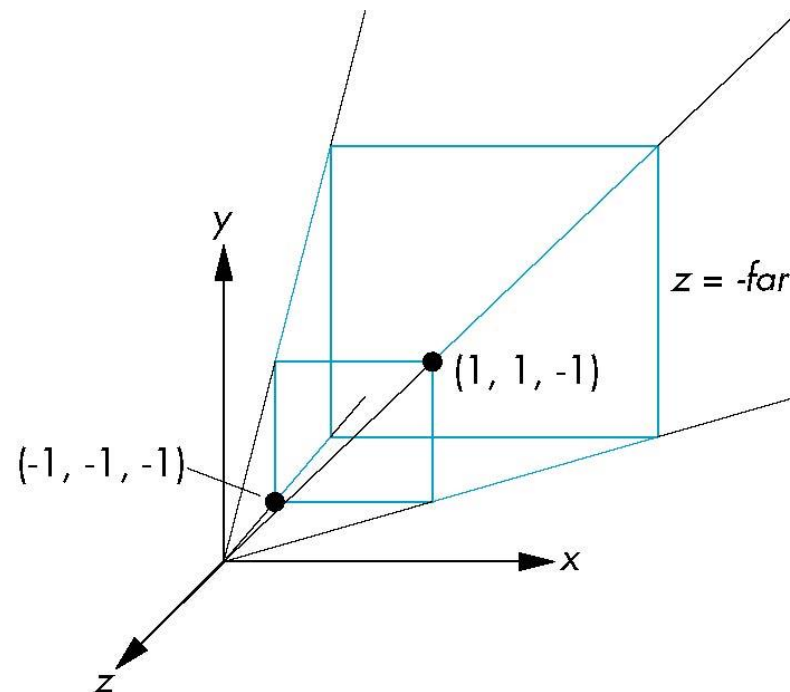
as seen from the top:



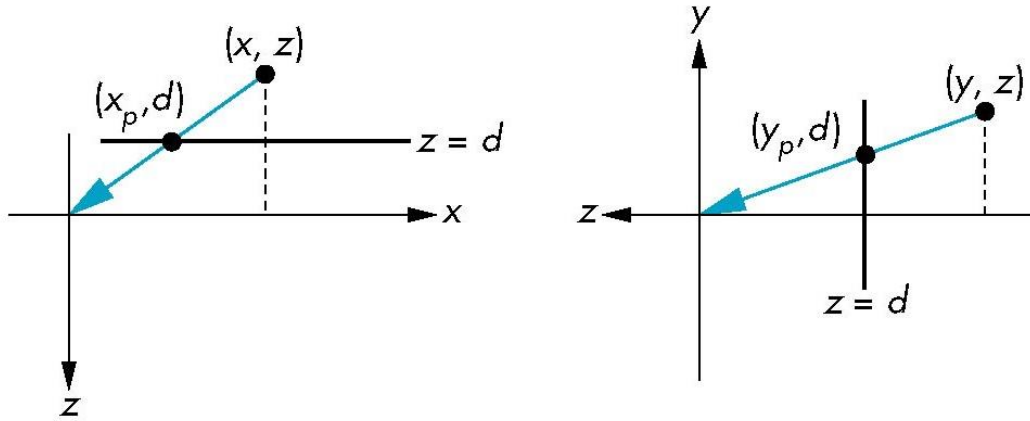
Simple Perspective

Consider a simple perspective with COP at the origin, the near clipping plane at $z = -1$, and a 90 degree field of view determined by the planes $x = \pm z$, $y = \pm z$

$$x = \pm z, \quad y = \pm z$$



Remember the Derivation of the Simple Perspective Projection Matrix



$$x_p = \frac{x}{z/d} \quad y_p = \frac{y}{z/d} \quad z_p = d$$

consider $\mathbf{q} = \mathbf{M}\mathbf{p}$ where $\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{bmatrix}$

$$\Rightarrow \text{for } \mathbf{p} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \Rightarrow \mathbf{q} = \begin{bmatrix} x \\ y \\ z \\ z/d \end{bmatrix}$$

Perspective Matrices

Resulting simple projection matrix (for $d = -1$) in homogeneous coordinates:

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Note that this matrix is independent of where the far clipping plane is

Generalization

$$\mathbf{N} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

after perspective division, the point $(x, y, z, 1)$ goes to

$$x'' = -x/z$$

$$y'' = -y/z$$

$$z'' = -(\alpha + \beta/z)$$

which projects orthogonally to the desired point regardless of α and β

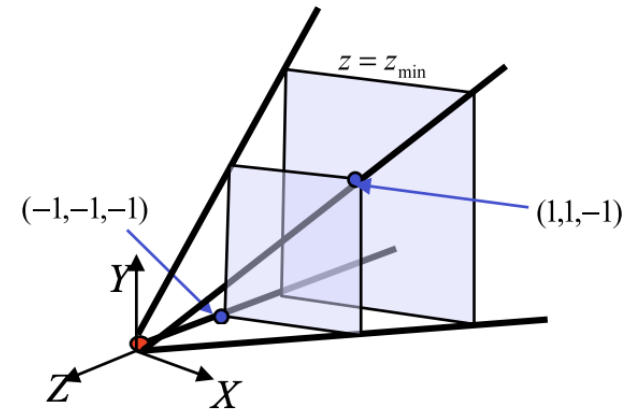
Simple Perspective Normalisation

A simple perspective projection with the projection plane at $-d$ on the z -axis can be represented as:

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1/d & 0 \end{bmatrix}$$

$$x_p = Cx_c$$

Setting $d=-1$ gives the semi-infinite view-volume bounded by the planes: $x_c = \pm z_c, y_c = \pm z_c$



We define a finite view frustum by: $z_{\min} < z_c < z_{\max}$

Consider the transformation N which transforms point $(x, y, z, 1)$ to (x', y', z', w) :

$$N = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

$$x' = x, y' = y$$

$$z' = \alpha z + \beta$$

$$w' = -z$$

In real co-ordinates:

$$x' = -x / z, y' = -y / z$$

$$z' = -(\alpha + \beta / z)$$

Simple Perspective Normalisation II

If we apply an orthographic projection to N we obtain:

$$C_{orth}N = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

which results in the projection: $x' = x, y' = y, z' = 0, w' = -z$

in real-coordinates this is: $x' = -x / z, y' = -y / z, z' = 0$

which is a perspective projection of the point for an image plane at $z = -1$

Thus we have achieved a perspective projection by combining the matrix N with an orthographic projection.

N is distorting the world space

How can we use N to normalise the projective view frustum to the canonical view frustum?

Simple Perspective Normalisation III

Matrix N is non-singular and transforms the original view-volume into a new view volume.

Choose α and β such that the new volume is the canonical clipping frustum

The sides of the projective view-volume after normalisation N are transformed to be the same as the canonic view-volume:

$$x_c = \pm z_c, y_c = \pm z_c$$
$$x' = Nx_c \Rightarrow x' = \pm 1, y' = \pm 1$$

The front and back are transformed to the planes:

$$z_c = z_{\min} \Rightarrow z' = -\left(\alpha + \frac{\beta}{z_{\min}}\right)$$
$$z_c = z_{\max} \Rightarrow z' = -\left(\alpha + \frac{\beta}{z_{\max}}\right)$$

Therefore, setting α and β we obtain the canonical view frustum:

$$\alpha = \frac{(z_{\max} + z_{\min})}{(z_{\max} - z_{\min})}, \beta = -\frac{2z_{\max}z_{\min}}{(z_{\max} - z_{\min})} \Rightarrow z' = \pm 1$$

Thus, we have transformed from a simple projective to canonical view-frustum

- This is a non-linear mapping which preserves the depth ordering

Picking α and β

If we pick

$$\alpha = \frac{\text{near} + \text{far}}{\text{far} - \text{near}}$$

$$\beta = \frac{2\text{near} * \text{far}}{\text{near} - \text{far}}$$

the near plane is mapped to $z = -1$

the far plane is mapped to $z = 1$

and the sides are mapped to $x = \pm 1, y = \pm 1$

Hence the new clipping volume is the default clipping volume

General Perspective Normalisation

In the simple perspective normalisation we assumed that the projection plane was at $z_c = -1$

We can generalise this to an arbitrary perspective projection

- (1) Convert the view-frustum to the simple symmetric projection frustum
- (2) Transform the simple frustum to the canonical view frustum

Conversion of an arbitrary asymmetric view frustum to the simple frustum:

- (i) Shear the frustum to obtain a symmetric frustum

$$\left(\frac{x_{\max} + x_{\min}}{2}, \frac{y_{\max} + y_{\min}}{2}, z_{\min} \right) \Rightarrow (0, 0, z_{\min})$$

keep in mind that $z_{\min}$ and $z_{\max}$ are negative

$$H(\cot \theta, \cot \phi) = H\left(\frac{x_{\max} + x_{\min}}{2 |z_{\min}|}, \frac{y_{\max} + y_{\min}}{2 |z_{\min}|} \right)$$

$$\Rightarrow x = \pm \left(\frac{x_{\max} - x_{\min}}{2 z_{\min}} \right), y = \pm \left(\frac{y_{\max} - y_{\min}}{2 z_{\min}} \right), z = z_{\max}, z = z_{\min}$$

$$H = \begin{bmatrix} 1 & 0 & \frac{(x_{\min} + x_{\max})}{2 |z_{\min}|} & 0 \\ 0 & 1 & \frac{(y_{\min} + y_{\max})}{2 |z_{\min}|} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- (ii) Scale the sides of the frustum

$$S\left(\frac{2 z_{\min}}{x_{\max} - x_{\min}}, \frac{2 z_{\min}}{y_{\max} - y_{\min}}, 1 \right)$$

$$\Rightarrow x = \pm z, y = \pm z$$

$$S = \begin{bmatrix} \frac{2 z_{\min}}{(x_{\max} - x_{\min})} & 0 & 0 & 0 \\ 0 & \frac{2 z_{\min}}{(y_{\max} - y_{\min})} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

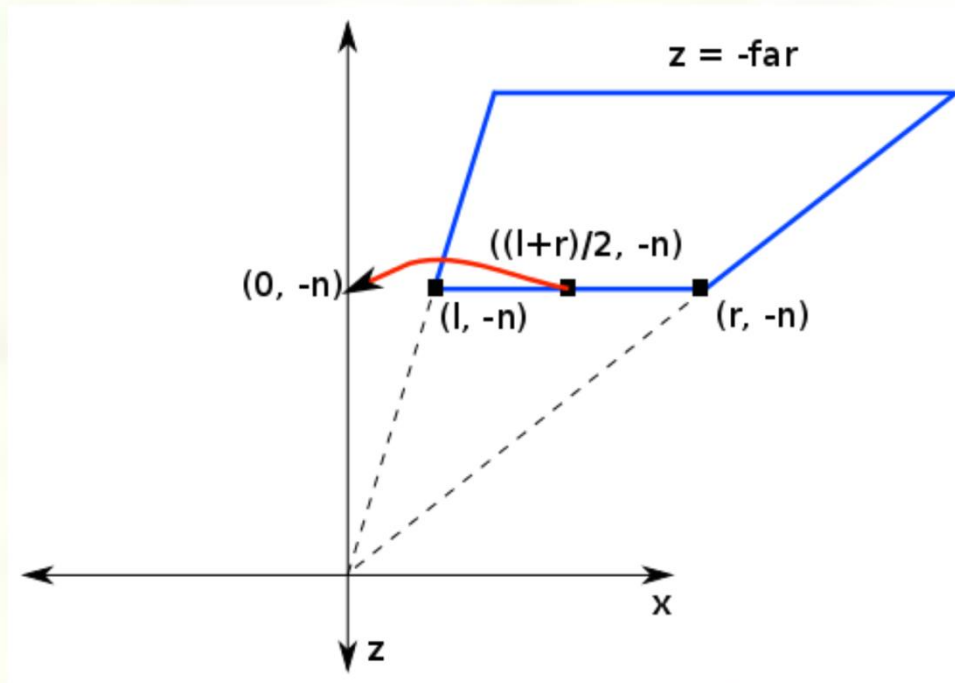
(visual aid for the shear)

$$x' = x + \frac{l+r}{2n}z$$

$$y' = y + \frac{t+b}{2n}z$$

$$z' = z$$

$$H = \begin{bmatrix} 1 & 0 & \frac{l+r}{2n} & 0 \\ 0 & 1 & \frac{t+b}{2n} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Step 1: Shear to center on $-z$ axis

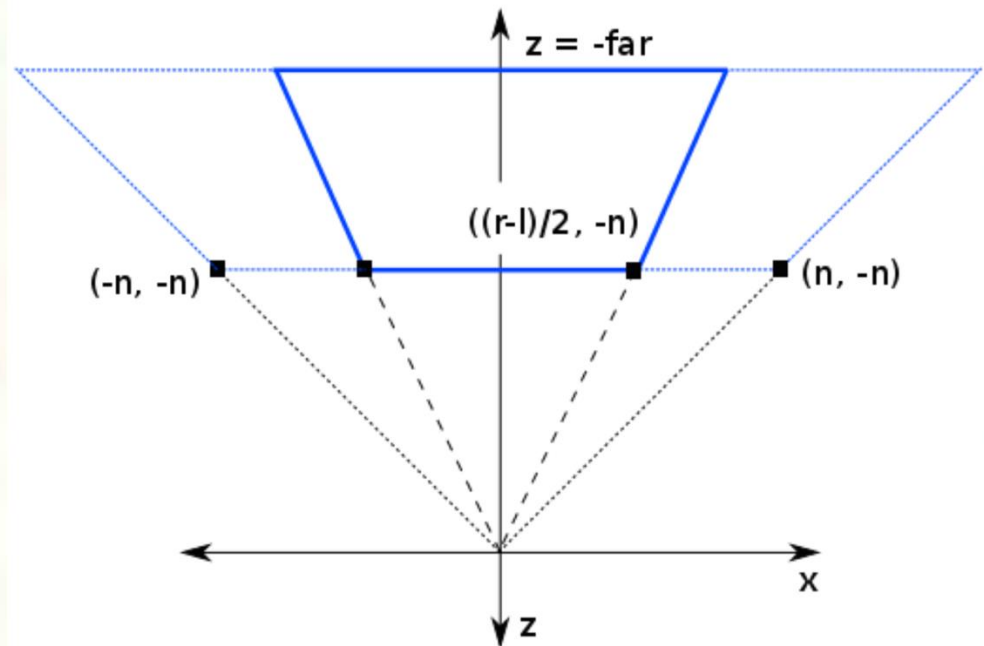
(visual aid for the scale)

$$x' = \frac{2n}{r-l}x$$

$$y' = \frac{2n}{t-b}y$$

$$z' = z$$

$$S = \begin{bmatrix} \frac{2n}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2n}{t-b} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Step 2: Scale so boundary slopes are ± 1

General Perspective Normalisation II

We can then concatenate the transform with the transformation of the simple view frustum to transform an arbitrary projective frustum to the canonical view frustum

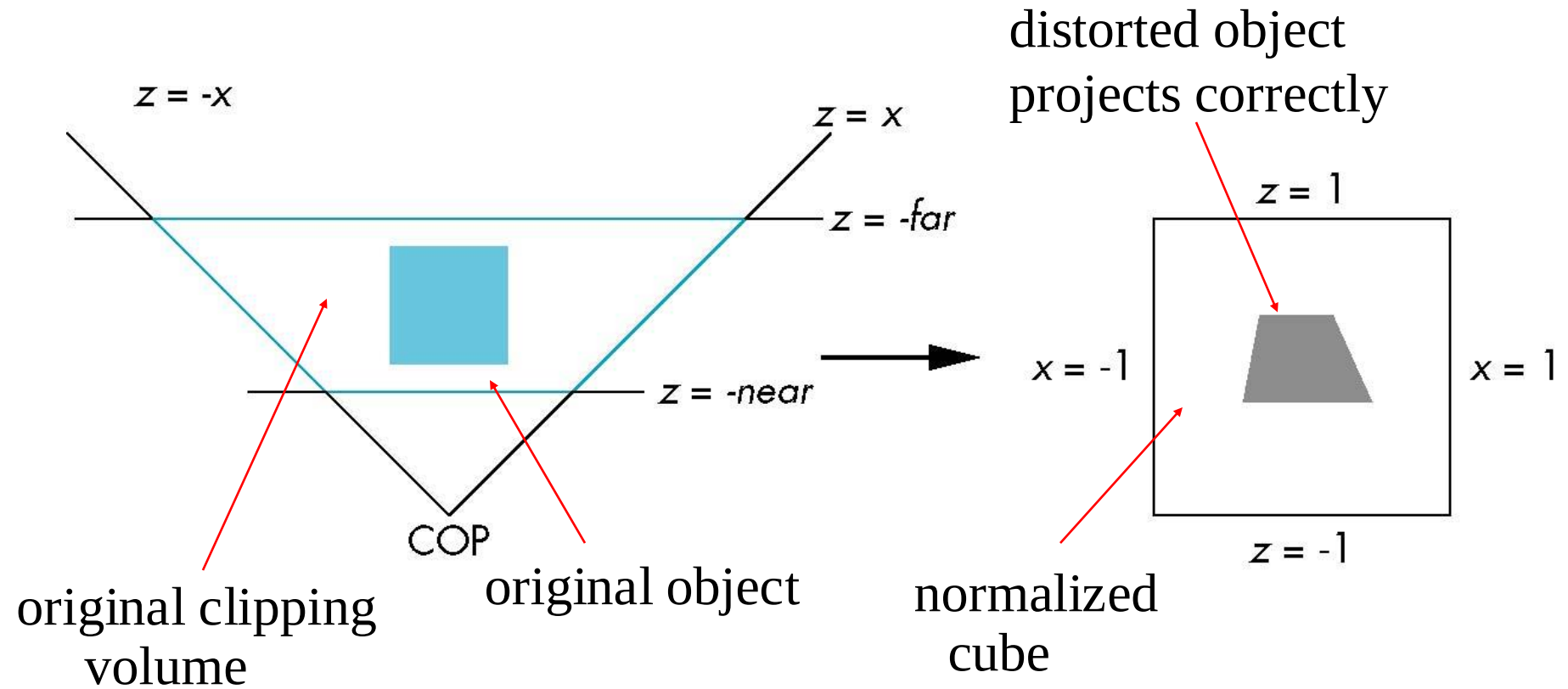
$$P = NSH = \begin{bmatrix} \frac{2z_{\min}}{(x_{\max} - x_{\min})} & 0 & \frac{(x_{\max} + x_{\min})}{(x_{\max} - x_{\min})} & 0 \\ 0 & \frac{2z_{\min}}{(y_{\max} - y_{\min})} & \frac{(y_{\max} + y_{\min})}{(y_{\max} - y_{\min})} & 0 \\ 0 & 0 & -\frac{(z_{\max} + z_{\min})}{(z_{\max} - z_{\min})} & -\frac{2z_{\max}z_{\min}}{(z_{\max} - z_{\min})} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

This can then be orthographically projected to obtain the original perspective projection

Hence, we can model any perspective projection as a transform to the canonic view frustum followed by an orthographic projection

The canonic view frustum is important for implementing visibility, shadowing

Normalization Transformation

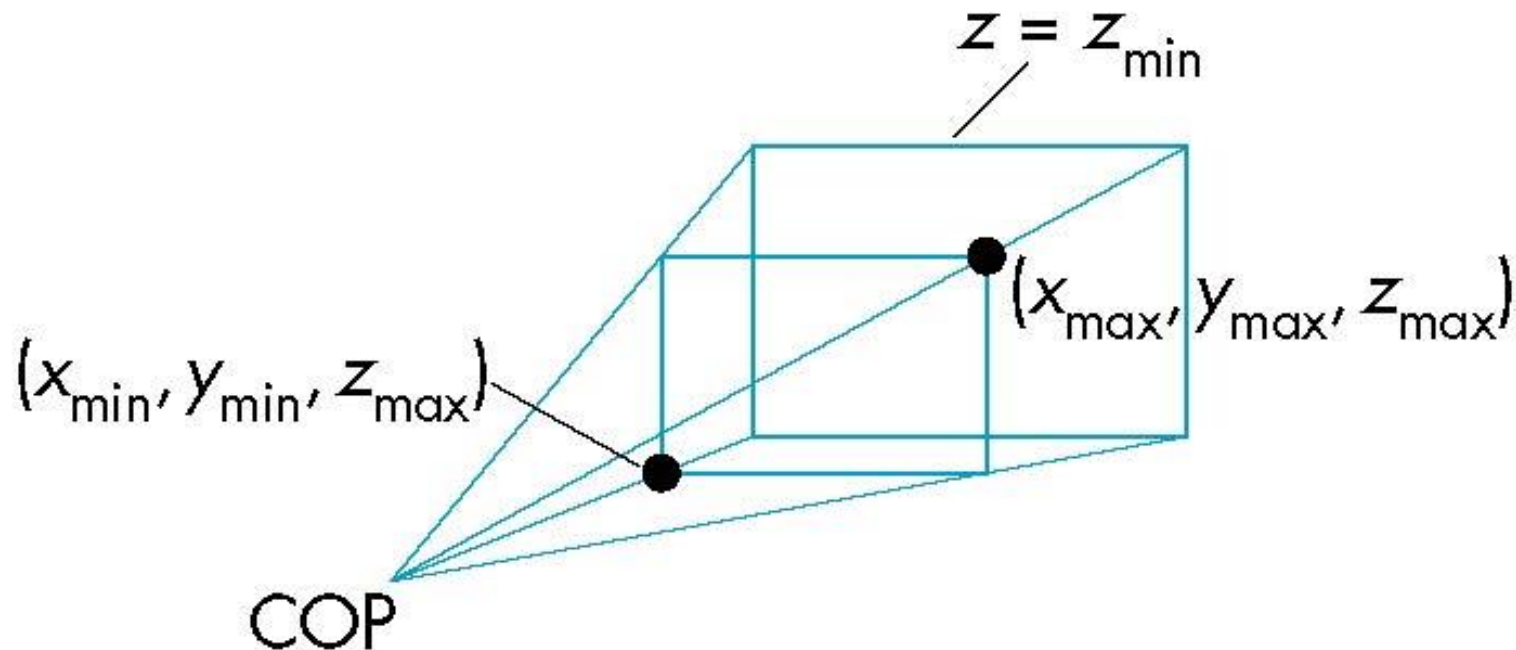


Normalization and Hidden-Surface Removal

- Although our selection of the form of the perspective matrices may appear somewhat arbitrary, it was chosen so that if $z_1 > z_2$ in the original clipping volume then the transformed points $z_1' > z_2'$
- Thus hidden surface removal works if we first apply the normalization transformation
- However, the formula $z'' = -(\alpha + \beta/z)$ implies that the distances are distorted by the normalization which can cause numerical problems especially if the near distance is small

WebGL Perspective

- **frustum** allows for an unsymmetrical viewing frustum (although **perspective** does not)



WebGL Perspective Matrix

- The normalization in **frustum** requires
 - an initial **shear** to form a right viewing pyramid (pyramid with apex on top of center)
 - followed by a **scaling** to get the normalized perspective volume
 - finally, the **perspective matrix**

$$\mathbf{P} = \mathbf{N} \mathbf{S} \mathbf{H}$$

The diagram illustrates the decomposition of the perspective matrix $\mathbf{P}$ into three components: $\mathbf{N}$ (normalization), $\mathbf{S}$ (scale), and $\mathbf{H}$ (shear). Red arrows point from the text labels below to the corresponding matrices in the equation above.

simple perspective
normalization

scale

shear

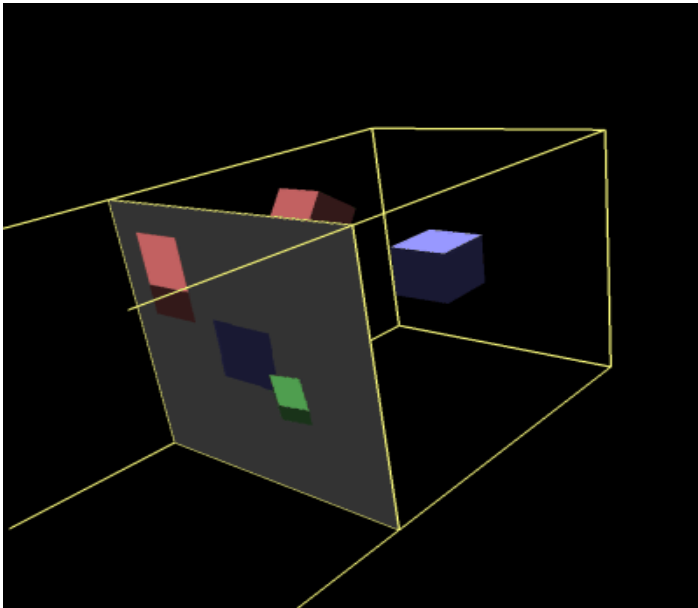
- This results in needing only a final orthogonal transformation

Why do we do it this way?

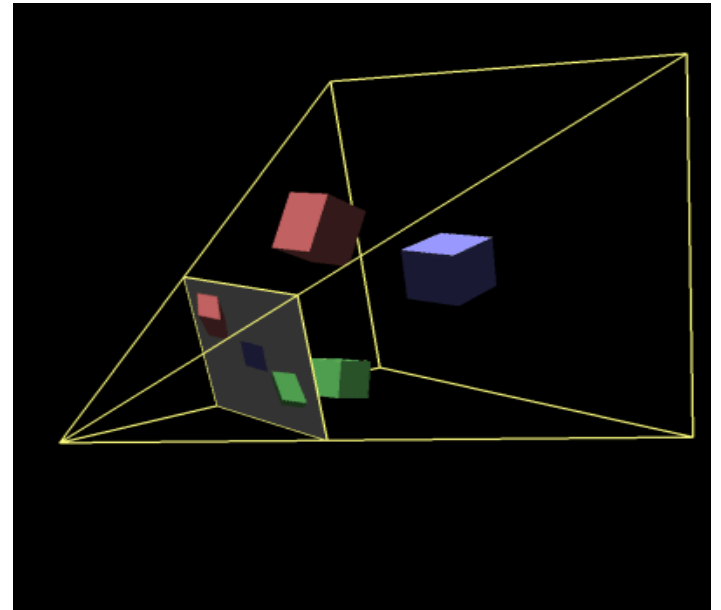
- Normalization allows for a single pipeline for both perspective and orthogonal viewing
- We stay in four dimensional homogeneous coordinates as long as possible to retain three-dimensional information needed for hidden-surface removal and shading
- We simplify clipping

Specifying What You Can See

Orthographic View



Perspective View



Classification of Transforms

Translation

Rotation

Rigid Body
preserves angles
and distances

Uniform Scaling

Similarity
preserves angles

Non-Uniform Scaling

Shear

Reflection

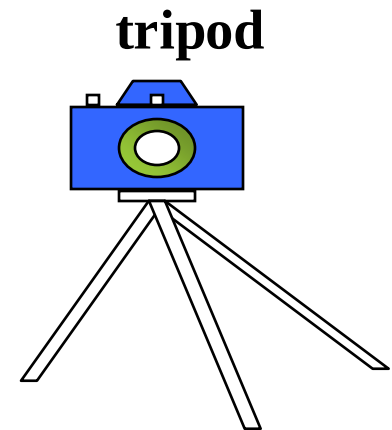
Affine
preserves
parallel lines

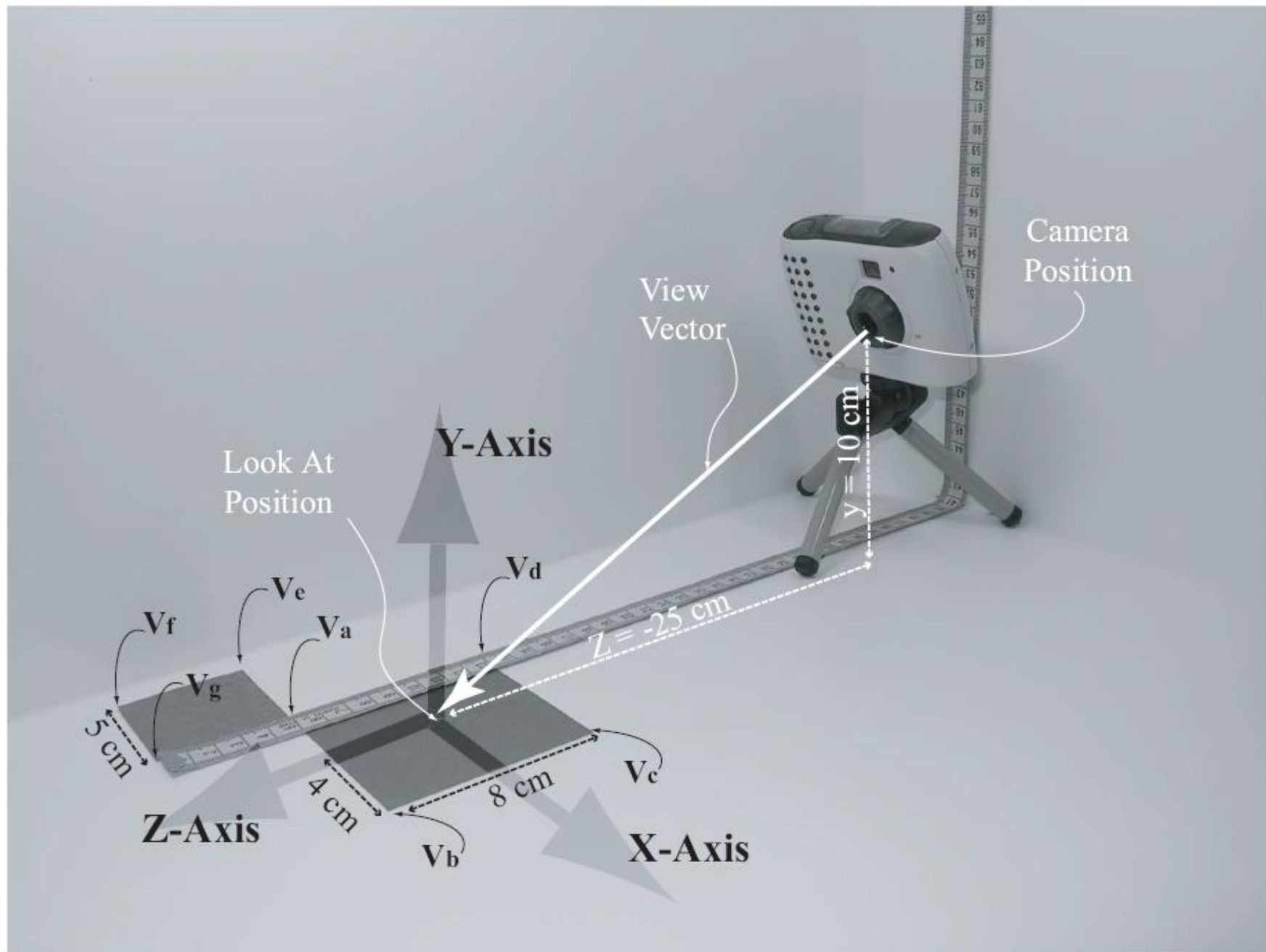
Perspective

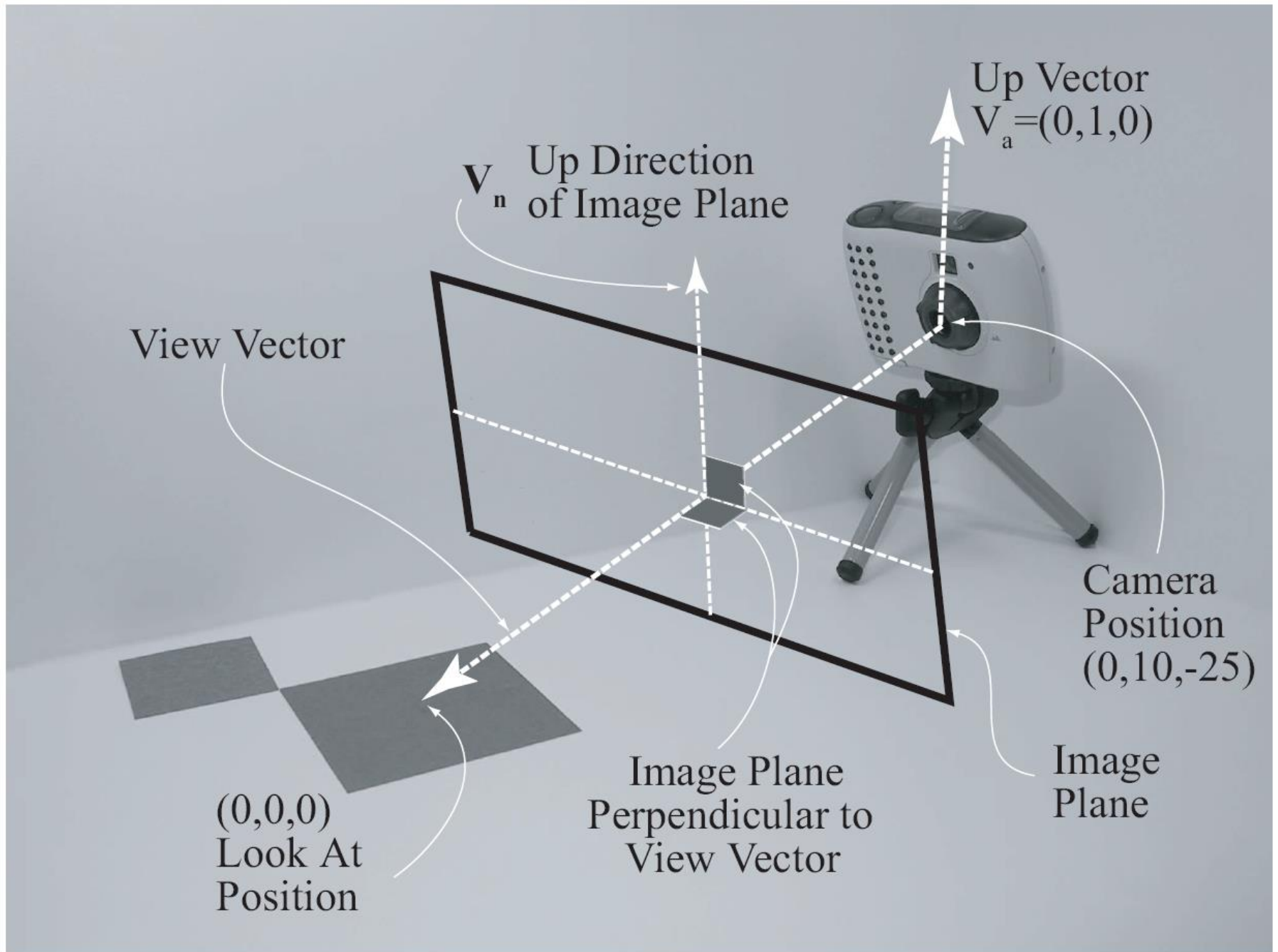
Projective preserves lines

Viewing Transformations

- Position the camera/eye in the scene
 - place the tripod down; aim camera
- To “fly through” a scene
 - change viewing transformation and redraw scene
- `lookAt( eyex, eyey, eyez, lookx, looky, lookz, upx, upy, upz )`
 - up vector determines unique orientation
 - careful of degenerate positions where the generated viewing matrix is undefined.





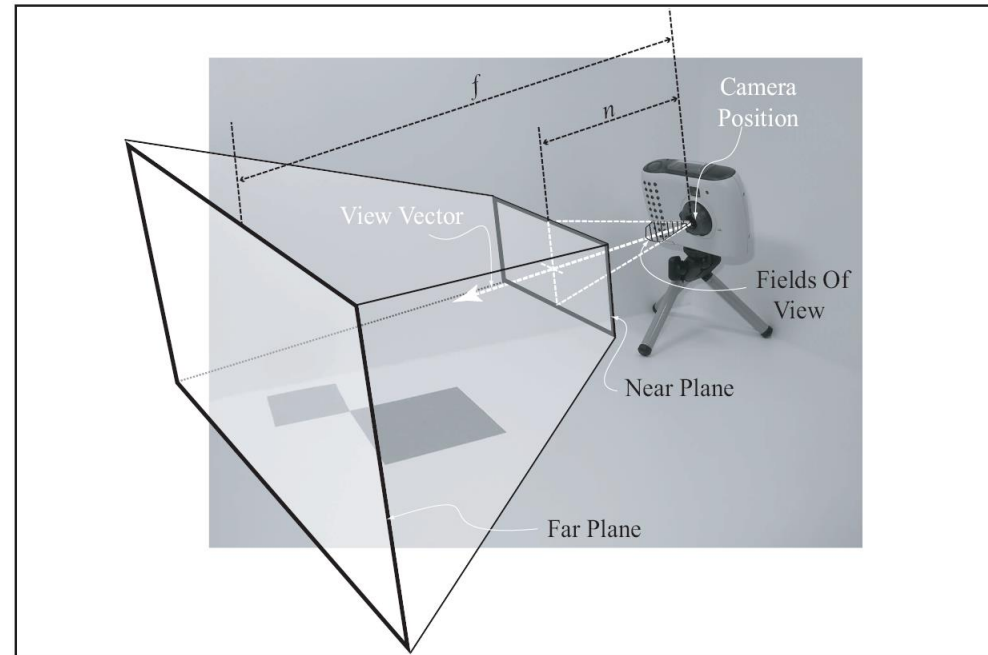


The Visible Volume

- Only geometries (primitives) *inside* the volume are visible
- All geometries (primitives) outside are ignored
- Primitives that straddle the volume are Clipped!

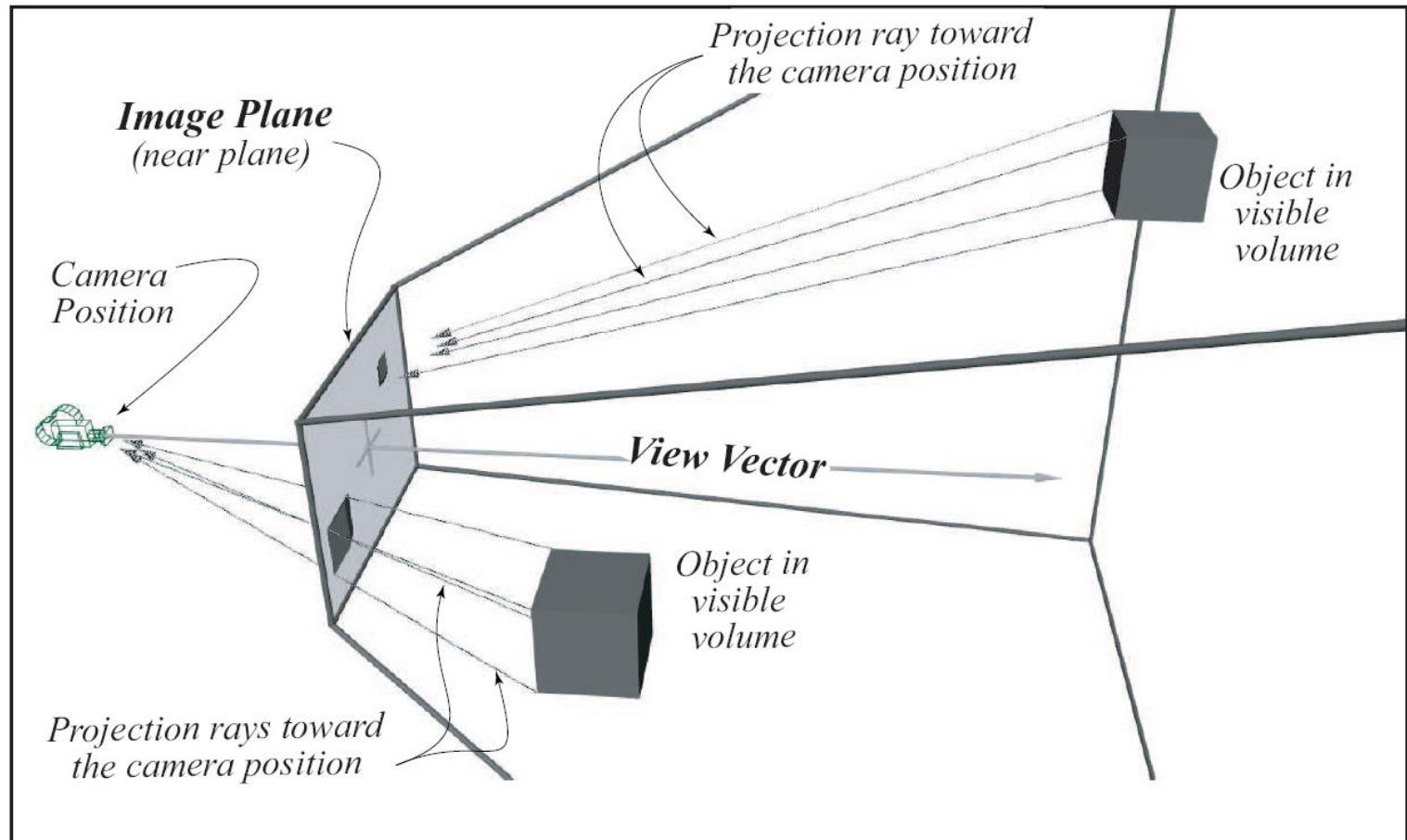
The Viewing Frustum Volume

- Volume defined by:
 - Near Plane (n)
 - Far Plane (f)
 - Field of view (fov)



- For Perspective Projection

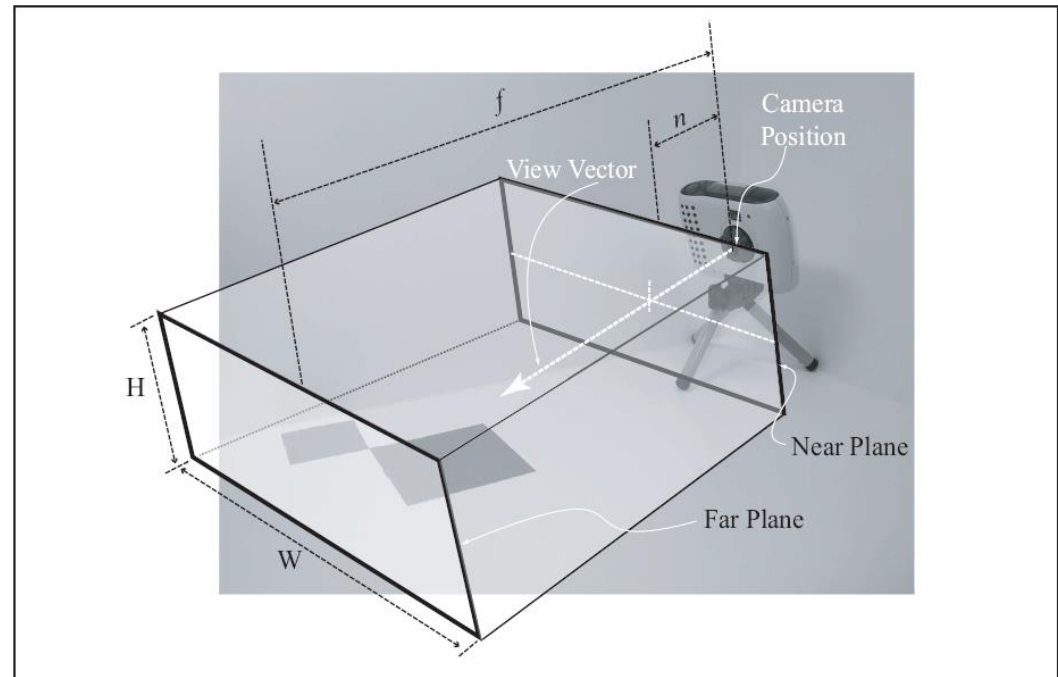
Perspective Projection



The Rectangular Visible Volume

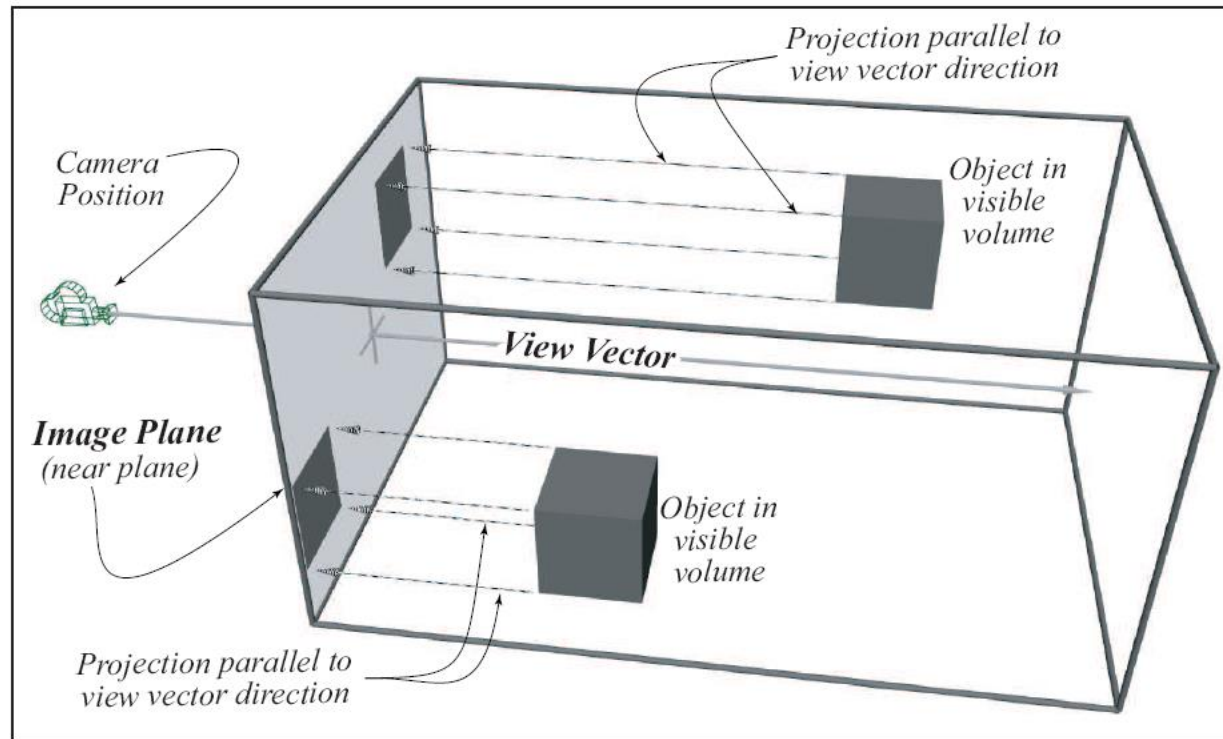
- Volume defined by:
 - Near Plane (n)
 - Far Plane (f)
 - Width (W)
 - Height (H)

$$\text{width} \times \text{height} \times \text{depth} = \\ W \times H \times (f - n)$$



- For Orthographic Projection

Orthographic Projection



Construction of the View Matrix

- LookAt function

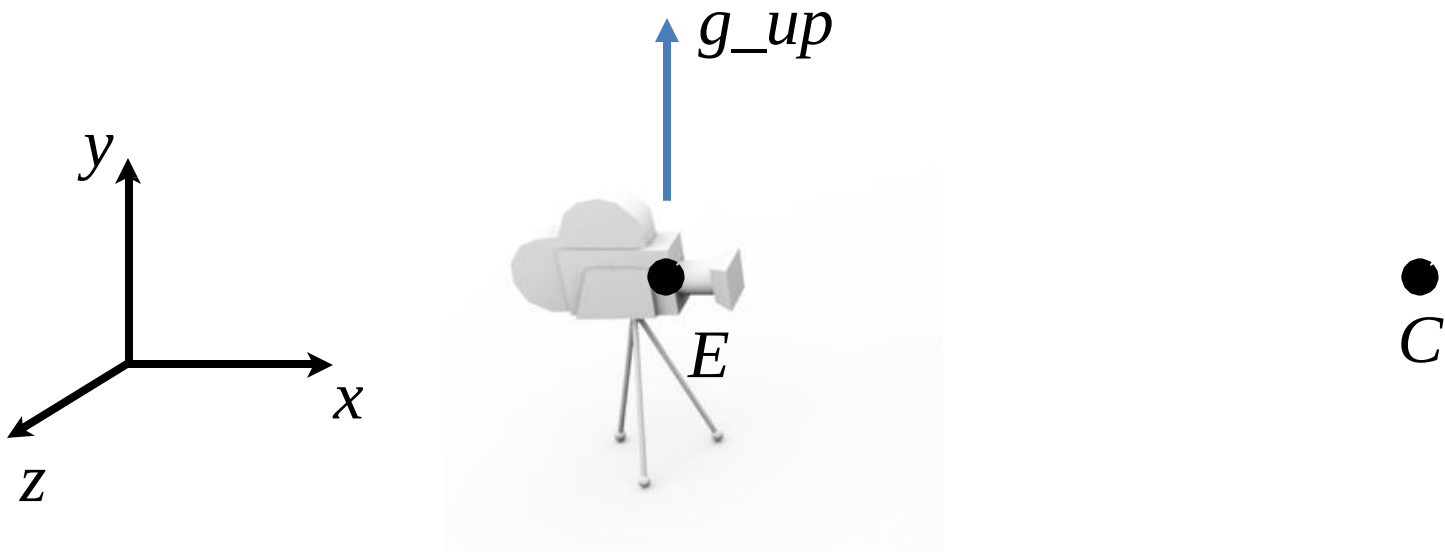
- Takes eye position (E), a position to look at (C) and an up vector (up)
- Constructs the **View** matrix, i.e., a matrix that transforms geometry (in world space) into the camera's coordinate system (camera space)

```
mat4 View = LookAt(E.x,E.y,E.z,           // Camera position
                  C.x,C.y,C.z,           // Center of interest
                  up.x, up.y, up.z);    // Up-vector
```



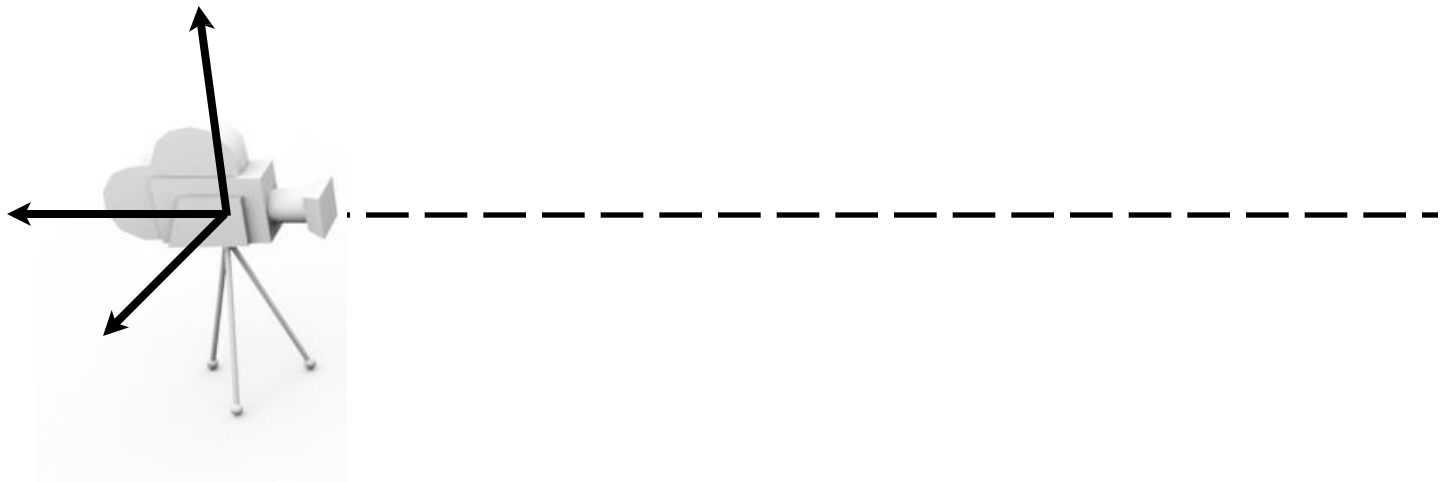
Camera Placement

- Specify camera position (E), center of interest (C) and global-up-vector (g_up)
[Global Up direction usually: $(0,1,0)$]



WebGL convention

- In WebGL: right-hand coordinate system, looking down $-z$.



Find orthonormal basis

- WebGL standard: camera looks along negative z.

Choose $\mathbf{w}$ in direction of $-(\mathbf{C} - \mathbf{E}) = \mathbf{E} - \mathbf{C}$

- The coordinate system of the camera is spanned by three orthonormal vectors $\{\mathbf{u}, \mathbf{v}, \mathbf{w}\}$ s.t.

camera-backward
vector

$$\mathbf{w} = \frac{\mathbf{E} - \mathbf{C}}{|\mathbf{E} - \mathbf{C}|}$$

camera-right
vector

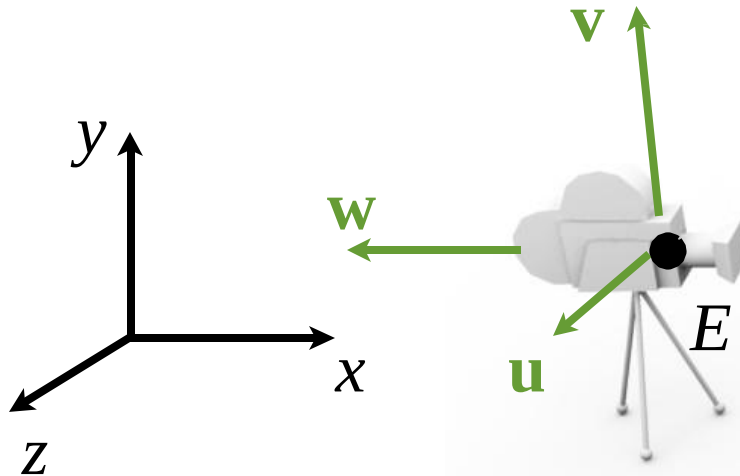
$$\mathbf{u} = \frac{g_up \times \mathbf{w}}{|g_up \times \mathbf{w}|}$$

camera-up
vector

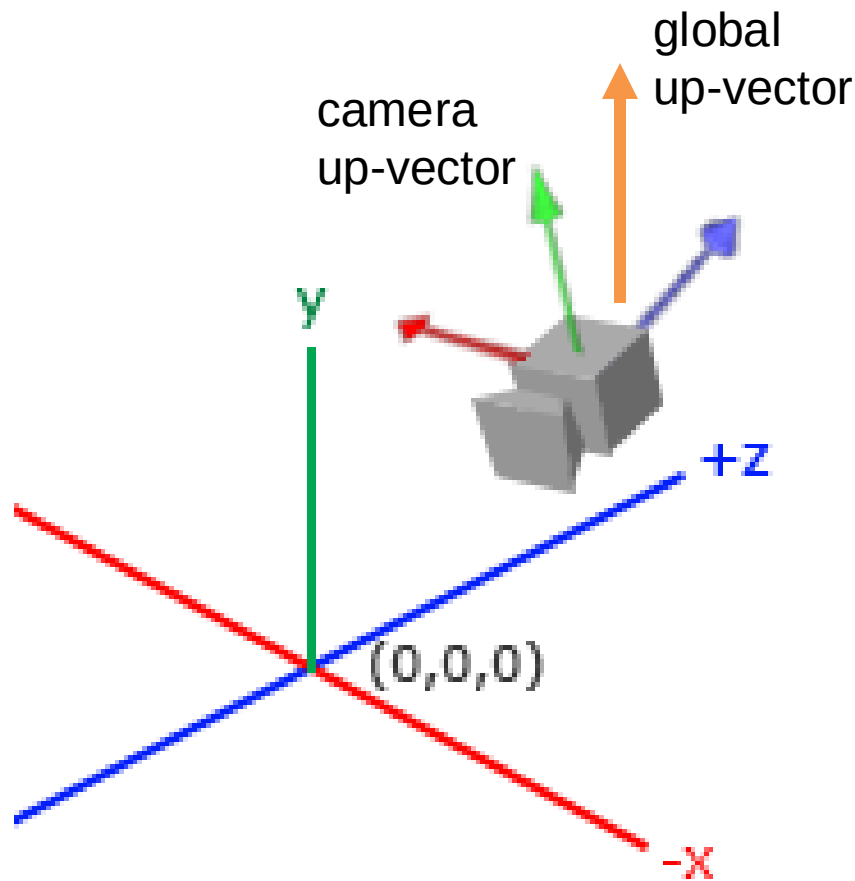
$$\mathbf{v} = \mathbf{w} \times \mathbf{u}$$

where $|\mathbf{X}|$ is the norm of $\mathbf{X}$

remember: $\mathbf{a} \times \mathbf{b} = \mathbf{c}$ is always
orthogonal to both $\mathbf{a}$ and $\mathbf{b}$

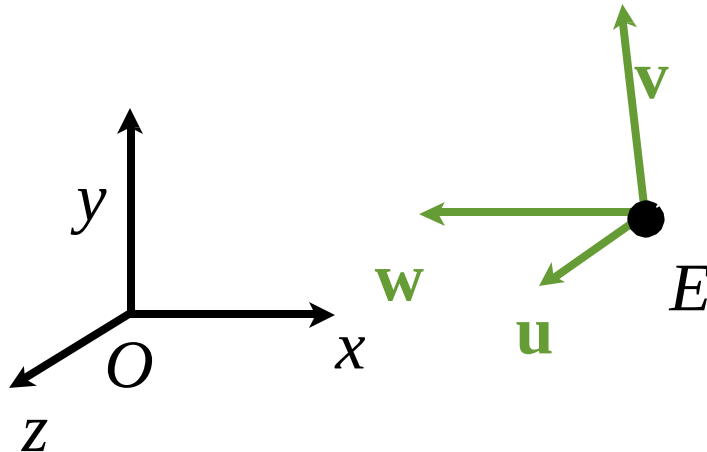


Global Up vs. Camera(Image Plane) Up



Find orthonormal basis

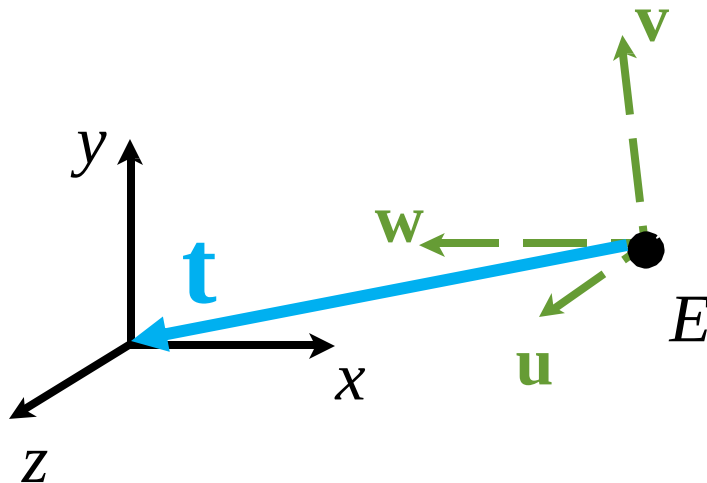
- Now, we look for matrix that transforms frame $\{\mathbf{u}, \mathbf{v}, \mathbf{w}, \mathbf{E}\}$ to $\{\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{O}\}$, the view (camera) frame
>> Translation and rotation



Find orthonormal basis

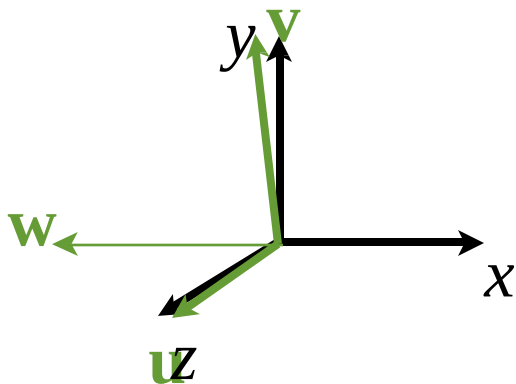
- Translate **uvwE** frame so that its origin coincides with the **xyzO** frame's

$$\mathbf{t} = \begin{bmatrix} -E_x \\ -E_y \\ -E_z \end{bmatrix}$$



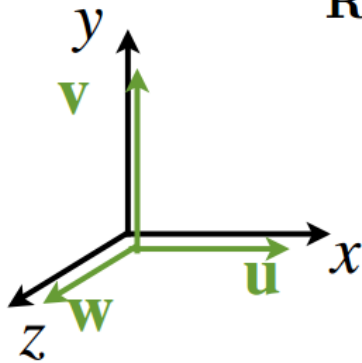
Find orthonormal basis

- Translate $uvwE$ frame so that its origin coincides with the $xyzO$ frame's



Find orthonormal basis

- Then rotate $\mathbf{uvw}$ basis so that the three axes align: $\mathbf{u} \parallel \mathbf{x}$, $\mathbf{v} \parallel \mathbf{y}$ and $\mathbf{w} \parallel \mathbf{z}$
- Rotation matrix given by $\mathbf{R} = \begin{bmatrix} - & \mathbf{u} & - \\ - & \mathbf{v} & - \\ - & \mathbf{w} & - \end{bmatrix}$
- $\mathbf{R}$ rotates vectors $\mathbf{uvw}$ to $\mathbf{xyz}$



$$\mathbf{R}\mathbf{u} = \begin{bmatrix} - & \mathbf{u} & - \\ - & \mathbf{v} & - \\ - & \mathbf{w} & - \end{bmatrix} \begin{bmatrix} | \\ \mathbf{u} \\ | \end{bmatrix} = \begin{bmatrix} \mathbf{u} \cdot \mathbf{u} \\ \mathbf{v} \cdot \mathbf{u} \\ \mathbf{w} \cdot \mathbf{u} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \mathbf{x}$$

$$\mathbf{R}\mathbf{v} = \mathbf{y}, \quad \mathbf{R}\mathbf{w} = \mathbf{z}$$

View Matrix

- Combine the two transforms
- The view matrix M:

1) move to center, and then 2) apply rotation

$$M = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ w_x & w_y & w_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -E_x \\ 0 & 1 & 0 & -E_y \\ 0 & 0 & 1 & -E_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotate

Move to center

$$\mathbf{w} = \frac{E - C}{|E - C|}$$

camera-forward
vector

$$\mathbf{u} = \frac{g_up \times w}{|g_up \times w|}$$

camera-right
vector

$$\mathbf{v} = \mathbf{w} \times \mathbf{u}$$

camera-up
vector

View Matrix

- Combine the two transforms
- The view matrix M:

1) move to center, and then 2) apply rotation

$$M = \begin{bmatrix} u_x & u_y & u_z & -(u \cdot E) \\ v_x & v_y & v_z & -(v \cdot E) \\ w_x & w_y & w_z & -(w \cdot E) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{w} = \frac{E - C}{|E - C|}$$

camera-forward
vector

$$\mathbf{u} = \frac{g_up \times w}{|g_up \times w|}$$

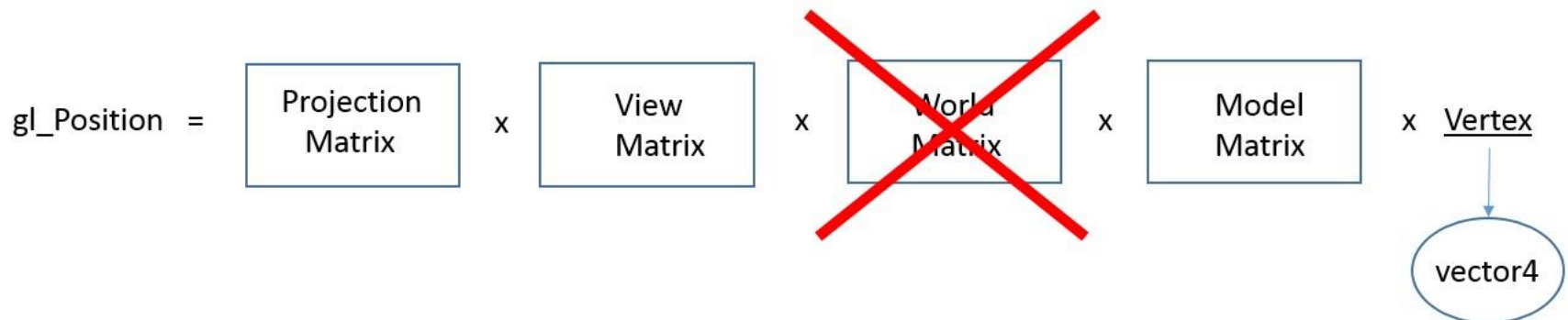
camera-right
vector

$$\mathbf{v} = \mathbf{w} \times \mathbf{u}$$

camera-up
vector

View Matrix

- View matrix simplifies our lives a little because we can transform our vertex coordinates directly to view space with it; so we don't care about the world space anymore
- Can regard the World matrix as an Identity matrix when you use View Matrix directly on vertex positions.



A WebGL program

Main

```
function main()
{
    const canvas = document.getElementById("glCanvas");
    const gl = canvas.getContext("webgl");
    if (!gl) {
        alert("Your browser does not support webgl!");
        return;
    }
    const vsSource = `...`;
    const fsSource = `...`;

    const programAndBuffer = init(gl, vsSource, fsSource);
    gl.useProgram(programAndBuffer.program);

    function render() {...}
}
```

Init - Setup Geometry

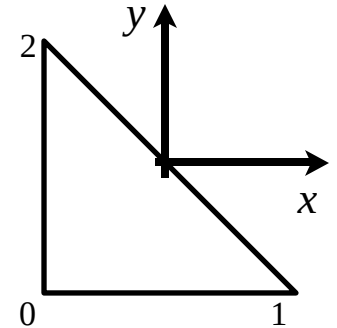
```
function init(gl, vsSource, fsSource)
{
    const glShaderProgramID = initProgram(gl, vsSource, fsSource);

    // Create geometry (one triangle)
    const vertices = [-0.5, -0.5, 1.0,
                     0.5, -0.5, 1.0,
                     -0.5, 0.5, 1.0];

    // Create a buffer object
    const _buffer = initBuffer(gl, vertices);

    // Initialize the vertex position attribute
    // from the vertex shader
    var pos = gl.getAttribLocation(glShaderProgramID, "vPosition" );
    gl.enableVertexAttribArray( pos );
    gl.vertexAttribPointer( pos, 3, gl.FLOAT, gl.FALSE, 0, 0 );

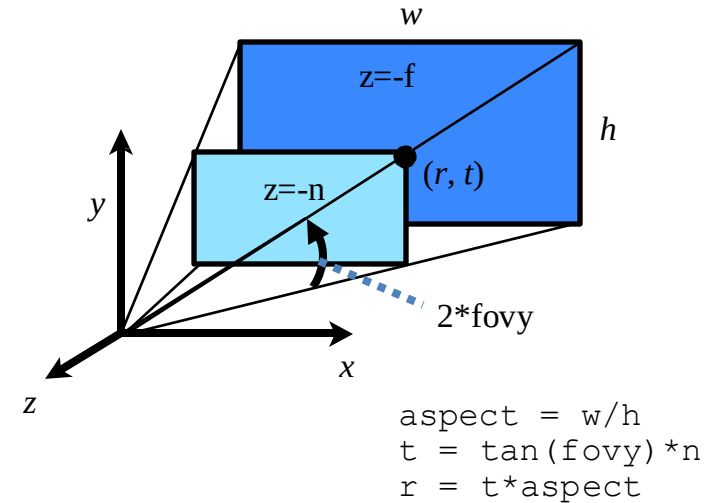
    return {
        program: glShaderProgramID,
        buffer: _buffer
    };
}
```



```
attribute vec4 vPosition;
uniform mat4 MVP;//ModelViewProj

void main()
{
    gl_Position = MVP*vPosition;
}
```

Resize



```
// If the size of the window changed,  
// call this to update the GL matrices
```

```
function resize(w, h)  
{
```

```
    if(h == 0) h = 1; // Prevent a divide by zero
```

```
    // calculate the projection matrix
```

```
    var aspect = w / h;
```

```
    var fovy = 45.0;
```

```
    var near = 0.01;
```

```
    var far = 10.0;
```

```
    projectionMatrix = perspective(fovy, aspect, near, far);
```

```
    gl.viewport(0, 0, w, h); // Set the viewport to be the entire  
    window
```

```
}
```

$$\begin{bmatrix} \frac{n}{r} & 0 & 0 & 0 \\ 0 & \frac{n}{t} & 0 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & \frac{-2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Render

```
function render()
{
    gl.clearColor(1.0,1.0,1.0,1.0);
    gl.clearDepth(1.0);
    gl.enable(gl.DEPTH_TEST);
    gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
    // Calculate the view matrix
    var at = vec3(0.0,0.0,0.0);
    var up = vec3(0.0,1.0,0.0);
    var View = lookAt(gEyePos, at, up );
    // Compute world matrix
    var world = ...;
    // Compute ModelViewProjection matrix
    var MVP = multiply(projectionMatrix,multiply(View, world));
    // Pass the modelview projection matrix to the shader
    var mvpID = gl.getUniformLocation(gShaderProgramID,"MVP");
    gl.uniformMatrix4fv(mvpID, false, toFloatArray(MVP));
    resize(gl.canvas.width, gl.canvas.height);
    // draw a triangle
    gl.drawArrays(gl.TRIANGLES, 0, 3);
    requestAnimationFrame(render);
}
```

```
attribute vec4 vPosition;
uniform mat4 MVP; //ModelViewProj

void main()
{
    gl_Position = MVP * vPosition;
}
```

Input Handling

```
// keyboard handling
function processKeys(event)
{
    switch (event.keyCode) {
        case 27:
            return;
        case 'w': case 'W':
            gEyePos.z -= 0.1;
            break;
        case 's': case 'S':
            gEyePos.z += 0.1;
            break;
        default:
            break;
    }
}
```